

ĐẠI HỌC BÁCH KHOA HÀ NỘI
KHOA TOÁN TIN

HỆ HỖ TRỢ QUYẾT ĐỊNH ĐẶT HÀNG
DỰA TRÊN DỰ BÁO CHUỖI THỜI GIAN ĐA PHÂN VỊ
TỐI ƯU HÓA TỒN KHO TRONG ĐIỀU KIỆN KHÔNG
CHẮC CHẮN

Môn học: Hệ hỗ trợ quyết định

Học kỳ: 20252 - Năm học 2024-2025

Thông tin sinh viên:

Họ và tên: Nguyễn Trường Giang, Đinh Hải Phong

MSSV: 20227044

Lớp: 169317

Email: giang.nt04@outlook.com.vn

Giảng viên hướng dẫn: Trần Ngọc Thăng, Lê Hải Hà

Hà Nội, tháng 6 năm 2025

TÓM TẮT

Báo cáo này trình bày thiết kế và triển khai một **Hệ hỗ trợ quyết định đặt hàng** (Decision Support System - DSS) tích hợp hai thành phần chính: (1) mô hình dự báo nhu cầu chuỗi thời gian đa phân vị và (2) mô hình tối ưu hóa tồn kho có cơ sở kinh tế.

Về dự báo, mô hình **RevIN + N-BEATS** được tự xây dựng bằng PyTorch kết hợp chuẩn hóa theo thực thể đảo ngược (Reversible Instance Normalization - RevIN) và kiến trúc N-BEATS với đầu ra đa phân vị (P5, P10, P25, P50, P75, P90, P95). Mô hình được huấn luyện bằng hàm mất mát *pinball loss* - tiêu chuẩn lý thuyết cho hồi quy phân vị - và tối ưu siêu tham số bằng thuật toán Optuna (100 thử nghiệm, TPE sampler). Dữ liệu là 120 quan sát tháng (2015-2024), chia theo chiến lược thời gian 8:1:1.

Trên tập kiểm tra (2024), mô hình đạt **MAE = 3.254**, **MAPE = 0,97%**, vượt trội gần **24 lần** so với phương pháp cơ sở Naive (MAE = 79.188, MAPE = 22,76%). Để kiểm chứng lựa chọn kiến trúc, chúng tôi **tự cài đặt và so sánh thêm bốn kiến trúc dự báo phân vị khác** (MLP, DLinear, TSMixer, NHITS) dưới cùng một giao thức huấn luyện: N-BEATS và NHITS cho kết quả tốt và ổn định nhất. Về quyết định tồn kho, kết quả phân vị được đưa vào **mô hình Newsvendor** - tỷ lệ tới hạn $CR = C_u / (C_u + C_o)$ xác định trực tiếp mức phân vị tối ưu cần chuẩn bị, mang ý nghĩa kinh tế rõ ràng mà dự báo điểm không thể hiện được. Mô hình **EOQ** bổ sung khuyến nghị cỡ lô cho vận hành dài hạn.

Hệ thống được triển khai dưới dạng hai ứng dụng web Streamlit: bản kỹ thuật đầy đủ chỉ số và bản dành cho người dùng kinh doanh với ngôn ngữ đời thường.

Từ khóa: dự báo chuỗi thời gian, N-BEATS, RevIN, hồi quy phân vị, pinball loss, newsvendor, tối ưu tồn kho, hệ hỗ trợ quyết định.

MỤC LỤC

Tóm tắt	1
1 Giới thiệu	8
1.1 Bối cảnh và động lực	8
1.2 Bài toán ra quyết định và người dùng	8
1.3 Mục tiêu hỗ trợ ra quyết định	9
1.4 Các câu hỏi quyết định cốt lõi	9
1.5 Phạm vi và đóng góp	10
1.6 Cấu trúc báo cáo	10
2 Cơ sở lý thuyết	11
2.1 Dự báo chuỗi thời gian	11
2.1.1 Bài toán dự báo điểm và dự báo phân vị	11
2.1.2 Hàm mất mát Pinball (Quantile Loss)	11
2.2 Reversible Instance Normalization (RevIN)	12
2.2.1 Vấn đề phân phối trôi dạt (Distribution Shift)	12
2.2.2 Cơ chế RevIN	12
2.3 N-BEATS	12
2.3.1 Kiến trúc tổng quan	12
2.3.2 Block N-BEATS	13
2.3.3 Ràng buộc đơn điệu phân vị (Monotone Quantile Head)	13
2.3.4 Doubly-Residual Stacking	13
2.4 Các kiến trúc dự báo phân vị khác (dùng để so sánh)	14
2.4.1 MLP thuần	14
2.4.2 DLinear	14
2.4.3 TSMixer	14
2.4.4 NHITS	15
2.5 Tối ưu siêu tham số bằng Optuna	15
2.6 Mô hình Newsvendor	15
2.6.1 Bài toán	15
2.6.2 Mức tồn kho tối ưu	15
2.6.3 Kết hợp với dự báo phân vị	16
2.7 Mô hình EOQ	16

3	Dữ liệu và Phương pháp	17
3.1	Mô tả dữ liệu	17
3.1.1	Tổng quan	17
3.1.2	Đặc điểm chuỗi	17
3.1.3	Chiến lược chia dữ liệu	17
3.2	Tiền xử lý dữ liệu	18
3.3	Phân tích khám phá dữ liệu (EDA)	18
3.3.1	Tổng quan chuỗi thời gian	18
3.3.2	Phân tích phân phối	19
3.3.3	Kiểm định tính dừng	20
3.3.4	Phân rã STL	20
3.3.5	Phân tích tự tương quan (ACF / PACF)	21
3.3.6	Phân tích mùa vụ theo tháng	22
3.3.7	Tương quan với biến ngoại sinh	22
3.3.8	Distribution Shift	23
3.3.9	Tóm tắt phát hiện EDA	25
3.4	Kiến trúc hệ thống	25
3.5	Thiết lập thực nghiệm	26
3.5.1	Dữ liệu huấn luyện	26
3.5.2	Không gian tìm kiếm siêu tham số	26
3.5.3	Quy trình huấn luyện cuối	26
3.5.4	Dự báo đệ quy 12 tháng	26
3.5.5	Các phương pháp baseline	27
3.5.6	Giao thức so sánh các kiến trúc	27
4	Kết quả và Thảo luận	28
4.1	Siêu tham số tốt nhất	28
4.2	Chỉ số hiệu suất dự báo	28
4.2.1	So sánh với baseline	28
4.2.2	Kết quả dự báo theo từng tháng	29
4.3	So sánh các kiến trúc mô hình	30
4.4	Hiệu chỉnh phân vị (Calibration)	32
4.5	Dự báo phân vị 12 tháng tới (2025)	34
4.6	Kết quả tối ưu hóa tồn kho (Newsvendor)	35
4.6.1	Ví dụ số: tháng 2025-01	35
4.6.2	Phân tích độ nhạy	36
4.7	Giao diện web	36
4.7.1	Bản kỹ thuật (app.py)	36

4.7.2	Bản dành cho người dùng kinh doanh (<code>app_business.py</code>) . .	37
4.8	Khuyến nghị ra quyết định	37
5	Kết luận	39
5.1	Tổng kết đóng góp	39
5.2	Hạn chế	39
5.3	Hướng phát triển	40
6	Phân công công việc	41
A	Cấu trúc mã nguồn	44
B	CODE SNIPPERS QUAN TRỌNG	45
C	Khai báo sử dụng AI	48
D	Bảng tự chấm điểm	49

DANH SÁCH HÌNH VẼ

3.1	Chuỗi thời gian Quantity và phân chia tập train/validation/test theo thời gian.	19
3.2	Histogram và Q-Q plot của chuỗi Quantity. Skewness = 0,377; Kurtosis = -0,716.	19
3.3	Phân rã STL chuỗi Quantity. Mùa vụ biên độ nhỏ; xu hướng biến động sau 2019.	21
3.4	ACF và PACF của chuỗi Quantity. Không có cấu trúc AR/MA rõ ràng.	22
3.5	Box plot phân phối Quantity theo từng tháng trong năm (gộp 2015-2024).	22
3.6	Hệ số tương quan Pearson giữa các biến ngoại sinh và Quantity. IPI và Imports có tương quan dương cao nhất.	23
3.7	Distribution shift rõ rệt: PromotionAmount tăng ~90 lần và CompetitorQuantity tăng ~4 lần trong 10 năm. Đường đứt: ranh giới train/val/test. . . .	24
3.8	Kiến trúc tổng thể của hệ hỗ trợ quyết định đặt hàng	25
4.1	So sánh MAE và MAPE giữa RevIN+N-BEATS và hai phương pháp baseline. Mô hình đề xuất vượt trội cả hai chỉ số với biên độ rất lớn. .	29
4.2	Dự báo phân vị (P10-P90, P25-P75, P50) so với giá trị thực tế trên tập test 2024. Đường thực tế nằm gần sát đường trung vị P50 trong hầu hết các tháng; khoảng P10-P90 bao phủ toàn bộ dao động thực tế.	30
4.3	So sánh MAE và MAPE của tất cả mô hình. Các mô hình học sâu (màu) vượt xa hai baseline Naive (xám). Thanh sai số = độ lệch chuẩn qua 3 hạt giống.	31
4.4	Chi tiết các mô hình học sâu: MAE, pinball loss và số tham số. DLinear/TSMixer cực gọn nhưng kém chính xác; N-BEATS/NHITS lớn hơn nhưng chính xác và ổn định hơn.	31
4.5	Dự báo trung vị (P50) của từng mô hình so với thực tế trên tập test 2024. N-BEATS, NHITS, MLP bám sát; DLinear và TSMixer lệch nhiều hơn ở các đỉnh (tháng 3, tháng 11).	32
4.6	Calibration: so sánh xác suất bao phủ lý tưởng (xanh) và thực tế trên tập test (đỏ). P75 bám chính xác; các phân vị cao bị undercover do tập test nhỏ (n=12).	33
4.7	Calibration của các kiến trúc trên tập test. MLP bám đường lý tưởng tốt nhất; N-BEATS bảo thủ (undercover) ở phân vị cao. Tập test nhỏ (n=12) nên đánh giá chỉ mang tính tham khảo.	34

- 4.8 Fan chart toàn bộ: 3 năm lịch sử gần nhất, tập test 2024 (thực tế), và dự báo phân vị 12 tháng tới (2025). Vùng đỏ đậm: P25-P75; vùng đỏ nhạt: P5-P95. 35
- 4.9 Đường cong chi phí kỳ vọng theo mức chuẩn bị hàng (tháng 01/2025, $C_u = 30,000đ$, $C_o = 4,000đ$). Điểm tối ưu Q^* tại CR= 0,882 (đường xanh lá) là nơi tổng chi phí đạt cực tiểu. 36

DANH SÁCH BẢNG

1.1	Các câu hỏi quyết định cốt lõi và nơi trả lời trong báo cáo	9
3.1	Chiến lược chia dữ liệu 8:1:1 theo thời gian	17
3.2	Các bước tiền xử lý dữ liệu	18
3.3	Thống kê mô tả chuỗi Quantity ($n = 120$ tháng)	19
3.4	Tóm tắt phát hiện EDA và lựa chọn phương pháp	25
3.5	Không gian tìm kiếm siêu tham số (Optuna, 100 trial)	26
4.1	Siêu tham số tốt nhất (Optuna, 100 trial)	28
4.2	Hiệu suất dự báo trên tập test (2024-01 $\rightarrow$ 2024-12)	28
4.3	So sánh dự báo P50 và thực tế tập test (2024)	29
4.4	So sánh hiệu suất các mô hình trên tập test (trung bình $\pm$ độ lệch chuẩn qua 3 hạt giống). Mô hình học sâu dùng cùng giao thức huấn luyện.	30
4.5	Calibration (coverage) trên tập test ($n = 12$)	32
4.6	Dự báo phân vị tháng 1/2025 (ví dụ)	34
4.7	Ảnh hưởng của cấu trúc chi phí đến quyết định đặt hàng (tháng 2025-01)	36
6.1	Bảng phân công công việc và đóng góp cá nhân	41
C.1	Khai báo công cụ AI đã sử dụng	48
D.1	Bảng nhóm tự đánh giá theo tiêu chí chấm điểm	49

CHƯƠNG 1. GIỚI THIỆU

1.1 Bối cảnh và động lực

Trong quản lý chuỗi cung ứng, câu hỏi “*Tháng này nên đặt bao nhiêu hàng?*” tưởng chừng đơn giản nhưng hàm chứa hai rủi ro đối lập: đặt ít thì hết hàng (mất doanh thu, mất khách), đặt nhiều thì tồn kho ế (ứ đọng vốn, hao hụt). Khó khăn căn bản nằm ở chỗ nhu cầu tương lai *không thể biết chắc* - dự báo tốt nhất cũng chỉ cung cấp một ước lượng kèm theo mức độ không chắc chắn.

Hầu hết các hệ thống thực tế vẫn dùng *dự báo điểm* (một con số duy nhất) rồi áp dụng quy tắc kinh nghiệm để tính an toàn tồn kho. Cách tiếp cận này bỏ qua một phần thông tin quan trọng: *phân phối xác suất* của nhu cầu, mà chính phân phối đó mới quyết định được mức tồn kho tối ưu về kinh tế.

Báo cáo này đề xuất một hệ thống khép kín kết hợp:

1. Dự báo **đa phân vị** (quantile forecasting) bằng RevIN + N-BEATS - cung cấp *cả phân phối* nhu cầu thay vì một con số.
2. Tối ưu hóa tồn kho bằng **mô hình Newsvendor** - dùng trực tiếp phân phối đó để tìm mức chuẩn bị tối ưu theo tiêu chí kinh tế.

1.2 Bài toán ra quyết định và người dùng

Bài toán thực tế: mỗi tháng, doanh nghiệp phải quyết định *đặt thêm bao nhiêu hàng* cho kỳ tới khi nhu cầu chưa biết chắc. Đây là một bài toán ra quyết định dưới điều kiện không chắc chắn, cân bằng giữa chi phí thiếu hàng (mất doanh thu) và chi phí thừa hàng (ứ đọng vốn).

Người sử dụng kết quả:

- *Quản lý kinh doanh / chủ cửa hàng* - người trực tiếp ra quyết định đặt hàng, cần con số khuyến nghị dễ hiểu kèm lý do.
- *Nhân viên phân tích / kế hoạch cung ứng* - người cần xem chi tiết dự báo, độ tin cậy và các chỉ số kỹ thuật để hiệu chỉnh.

Quyết định cần hỗ trợ: (1) lượng hàng nên chuẩn bị cho tháng tới ứng với khẩu vị rủi ro/cấu trúc chi phí của doanh nghiệp (Newsvendor); (2) cỡ lô và điểm đặt hàng lại cho vận hành dài hạn (EOQ).

1.3 Mục tiêu hỗ trợ ra quyết định

1. Xây dựng mô hình dự báo chuỗi thời gian đa phân vị tự triển khai (RevIN + N-BEATS, PyTorch), tối ưu siêu tham số bằng Optuna; **so sánh với nhiều kiến trúc khác** để biện minh lựa chọn.
2. Kết nối đầu ra dự báo với mô hình Newsvendor để biến dự báo thành *khuyến nghị đặt hàng cụ thể* có ý nghĩa kinh tế.
3. Triển khai hệ thống dưới dạng ứng dụng web tương tác (Streamlit) phục vụ hai nhóm người dùng: kỹ thuật viên phân tích và quản lý kinh doanh.

1.4 Các câu hỏi quyết định cốt lõi

Bảng 1.1 tóm tắt câu trả lời cho các câu hỏi cốt lõi của một bài toán hỗ trợ ra quyết định; chi tiết được triển khai ở các chương sau.

Bảng 1.1: Các câu hỏi quyết định cốt lõi và nơi trả lời trong báo cáo

Câu hỏi	Trả lời ngắn gọn	Mục
Bài toán thực tế là gì?	Đặt bao nhiêu hàng mỗi tháng dưới nhu cầu bất định	1.2
Ai dùng kết quả?	Quản lý kinh doanh và nhân viên phân tích cung ứng	1.2
Người dùng cần quyết định gì?	Lượng đặt thêm (Newsvendor) + cỡ lô/ROP (EOQ)	1.2
Dữ liệu nào được dùng?	120 quan sát tháng doanh số (2015-2024) + 10 biến vĩ mô	Ch. 3
Xử lý & phân tích thế nào?	Làm sạch, RevIN, EDA, dự báo phân vị, tối ưu tồn kho	Ch. 3
Kết quả quan trọng nhất?	MAPE 0,97% (test); N-BEATS tốt nhất khi so sánh	Ch. 4
Đề xuất hành động gì?	Mức đặt hàng theo phân vị tới hạn CR	Ch. 4, 4.8
Hạn chế/rủi ro/điều kiện?	Tập test nhỏ, dự báo đệ quy, đơn biến	Ch. 5

1.5 Phạm vi và đóng góp

- **Dữ liệu:** chuỗi thời gian doanh số tháng thực tế, 120 quan sát (2015-2024), cùng 10 biến ngoại sinh vĩ mô.
- **Đóng góp kỹ thuật:** (i) đầu ra đơn điệu đa phân vị qua tham số hóa softplus-cumsum, dùng chung cho mọi kiến trúc; (ii) chuẩn hóa RevIN 3D tương thích với tensor đầu ra ($\text{batch} \times \text{horizon} \times Q$); (iii) **tự cài đặt và so sánh 5 kiến trúc** dự báo phân vị (MLP, DLinear, TSMixer, NHITS, N-BEATS) dưới cùng giao thức; (iv) tích hợp mô hình Newsvendor từ phân vị rời rạc qua nội suy.
- **Đóng góp ứng dụng:** hai giao diện web phục vụ hai đối tượng với ngôn ngữ và mức độ chi tiết khác nhau.

1.6 Cấu trúc báo cáo

Báo cáo gồm 6 chương: Chương 2 trình bày cơ sở lý thuyết (các mô hình dự báo và tối ưu tồn kho); Chương 3 mô tả dữ liệu, tiền xử lý và phương pháp; Chương 4 trình bày kết quả thực nghiệm, *so sánh các mô hình* và khuyến nghị quyết định; Chương 5 nêu hạn chế và định hướng phát triển; Chương 6 trình bày phân công công việc.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1 Dự báo chuỗi thời gian

2.1.1 Bài toán dự báo điểm và dự báo phân vị

Cho chuỗi thời gian $y_1, y_2, \dots, y_T$. Dự báo điểm tìm hàm $\hat{y}_{T+h} = f(y_1, \dots, y_T)$ nhằm tối thiểu hóa một sai số như MAE hoặc MSE. Dự báo phân vị bậc $q \in (0, 1)$ tìm $\hat{F}^{-1}(q)$ sao cho xác suất thực tế $y_{T+h} \leq \hat{F}^{-1}(q)$ xấp xỉ q :

$$P(y_{T+h} \leq \hat{F}^{-1}(q)) \approx q. \quad (2.1)$$

Dự báo đồng thời nhiều phân vị cho phép ước lượng *toàn bộ phân phối* nhu cầu mà không cần giả thiết tham số (Gaussian, v.v.).

2.1.2 Hàm mất mát Pinball (Quantile Loss)

Để huấn luyện mô hình dự báo phân vị, hàm mất mát pinball (còn gọi là quantile loss) được định nghĩa [3]:

$$\mathcal{L}_q(\hat{y}, y) = \max[q(y - \hat{y}), (q - 1)(y - \hat{y})], \quad (2.2)$$

hay tương đương:

$$\mathcal{L}_q(\hat{y}, y) = \begin{cases} q(y - \hat{y}) & \text{nếu } y \geq \hat{y}, \\ (q - 1)(y - \hat{y}) & \text{nếu } y < \hat{y}. \end{cases} \quad (2.3)$$

Với tập phân vị $\mathcal{Q} = \{q_1, \dots, q_K\}$, tổng pinball loss trung bình:

$$\mathcal{L} = \frac{1}{NK} \sum_{i=1}^N \sum_{k=1}^K \mathcal{L}_{q_k}(\hat{y}_{i,q_k}, y_i). \quad (2.4)$$

Hàm này phạt bất đối xứng theo q : phân vị cao (q lớn) phạt nặng khi dự báo *thấp hơn* thực tế, và ngược lại.

2.2 Reversible Instance Normalization (RevIN)

2.2.1 Vấn đề phân phối trôi dạt (Distribution Shift)

Chuỗi thời gian thực tế thường có phân phối thay đổi theo thời gian - xu hướng, mùa vụ, hoặc thay đổi cấu trúc (structural break). Mô hình học trên dữ liệu cũ $(\mu_{\text{train}}, \sigma_{\text{train}})$ sẽ gặp khó khăn khi dự báo giai đoạn mới có $\mu_{\text{test}} \neq \mu_{\text{train}}$.

2.2.2 Cơ chế RevIN

RevIN [2] chuẩn hóa từng cửa sổ đầu vào (instance) theo mean/std của chính nó, thay vì dùng thống kê toàn cục:

Bước chuẩn hóa (normalize):

$$\hat{x}_t = \frac{x_t - \hat{\mu}}{\hat{\sigma} + \varepsilon}, \quad \hat{\mu} = \frac{1}{L} \sum_{t=1}^L x_t, \quad \hat{\sigma} = \sqrt{\frac{1}{L} \sum_{t=1}^L (x_t - \hat{\mu})^2 + \varepsilon}. \quad (2.5)$$

Với tham số affine học được γ, β (tùy chọn):

$$\tilde{x}_t = \gamma \cdot \hat{x}_t + \beta. \quad (2.6)$$

Bước đảo ngược (denormalize): Sau khi mạng tạo dự báo $\hat{y}$ ở không gian chuẩn hóa, chuyển về thang đo gốc:

$$y = \hat{y} \cdot \hat{\sigma} + \hat{\mu}. \quad (2.7)$$

Với đầu ra đa phân vị ($\text{batch} \times H \times Q$), phép denormalize mở rộng:

$$\mathbf{Y}_{b,h,q} = \hat{\mathbf{Y}}_{b,h,q} \cdot \hat{\sigma}_b + \hat{\mu}_b, \quad (2.8)$$

trong đó $\hat{\sigma}_b, \hat{\mu}_b$ được broadcast theo chiều batch.

2.3 N-BEATS

2.3.1 Kiến trúc tổng quan

N-BEATS [1] là mạng nơ-ron thuần (fully-connected) không hồi quy theo chuỗi (recurrence-free), gồm nhiều *stack*, mỗi *stack* gồm nhiều *block*. Kiến trúc sử dụng cơ chế **doubly-residual stacking**: mỗi block nhận tín hiệu dư từ block trước, trừ đi

backcast và cộng dồn forecast.

2.3.2 Block N-BEATS

Mỗi block là một MLP gồm n_{layers} lớp ẩn với ReLU:

$$\mathbf{h}_1 = \text{ReLU}(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1), \quad (2.9)$$

$$\mathbf{h}_l = \text{ReLU}(\mathbf{W}_l \mathbf{h}_{l-1} + \mathbf{b}_l), \quad l = 2, \dots, n_{\text{layers}}. \quad (2.10)$$

Block xuất ra hai đầu:

$$\hat{\mathbf{x}}_{\text{back}} = \mathbf{W}_{\text{back}} \mathbf{h}, \quad \in \mathbb{R}^L \quad (\text{backcast}), \quad (2.11)$$

$$\hat{\mathbf{y}}_{\text{fore}} = \mathbf{W}_{\text{fore}} \mathbf{h}, \quad \in \mathbb{R}^{H \times K} \quad (\text{forecast đa phân vị}). \quad (2.12)$$

2.3.3 Ràng buộc đơn điệu phân vị (Monotone Quantile Head)

Để đảm bảo $\hat{F}^{-1}(q_1) \leq \hat{F}^{-1}(q_2)$ với mọi $q_1 < q_2$ (không bị *quantile crossing*), đầu ra được tham số hóa qua cumsum-softplus [8]:

$$\hat{y}_{h,0} = \mathbf{W}_0 \mathbf{h} \quad (\text{phân vị thấp nhất, P5}), \quad (2.13)$$

$$\hat{y}_{h,k} = \hat{y}_{h,0} + \sum_{j=1}^k \text{softplus}(\mathbf{W}_j \mathbf{h}), \quad k = 1, \dots, K-1. \quad (2.14)$$

Vì $\text{softplus}(z) > 0$ với mọi z , cumsum luôn tăng đơn điệu, đảm bảo $P5 \leq P10 \leq \dots \leq P95$.

2.3.4 Doubly-Residual Stacking

Với n_{blocks} block trong một stack:

$$\mathbf{x}^{(1)} = \mathbf{x}_{\text{in}}, \quad (2.15)$$

$$\mathbf{x}^{(l+1)} = \mathbf{x}^{(l)} - \hat{\mathbf{x}}_{\text{back}}^{(l)}, \quad (2.16)$$

$$\hat{\mathbf{Y}}_{\text{stack}} = \sum_{l=1}^{n_{\text{blocks}}} \hat{\mathbf{y}}_{\text{fore}}^{(l)}. \quad (2.17)$$

Qua n_{stacks} stack tương tự:

$$\hat{\mathbf{Y}}_{\text{final}} = \sum_{s=1}^{n_{\text{stacks}}} \hat{\mathbf{Y}}_s. \quad (2.18)$$

2.4 Các kiến trúc dự báo phân vị khác (dùng để so sánh)

Để tiện minh lựa chọn N-BEATS, chúng tôi tự cài đặt thêm bốn kiến trúc dự báo phân vị và so sánh dưới cùng một giao thức (Chương 4.3). Tất cả đều bao bọc bằng RevIN, dùng chung *đầu ra đơn điệu softplus-cumsum* và huấn luyện bằng pinball loss; chỉ khác nhau ở **phần lõi** (backbone).

2.4.1 MLP thuần

Mô hình học sâu cơ bản: một perceptron nhiều lớp ánh xạ trực tiếp cửa sổ đầu vào $\mathbf{x} \in \mathbb{R}^L$ sang đầu ra $H \times K$ phân vị, không có cấu trúc chuyên biệt cho chuỗi thời gian. Dùng để kiểm tra xem các kiến trúc phức tạp hơn có thực sự cải thiện hay không.

2.4.2 DLinear

DLinear [11] phân rã chuỗi đầu vào thành hai thành phần rồi áp một lớp tuyến tính riêng cho mỗi thành phần:

$$\mathbf{x}_{\text{trend}} = \text{AvgPool}_k(\mathbf{x}), \quad \mathbf{x}_{\text{season}} = \mathbf{x} - \mathbf{x}_{\text{trend}}, \quad (2.19)$$

$$\hat{\mathbf{Y}} = \mathbf{W}_{\text{trend}} \mathbf{x}_{\text{trend}} + \mathbf{W}_{\text{season}} \mathbf{x}_{\text{season}}. \quad (2.20)$$

Đây là mô hình cực kỳ gọn nhẹ (chỉ vài trăm tham số) nhưng là baseline mạnh cho nhiều bài toán dự báo dài hạn.

2.4.3 TSMixer

TSMixer [12] là kiến trúc toàn-MLP xen kẽ hai phép trộn trên tensor ($\text{time} \times \text{feature}$), mỗi phép có kết nối dư và chuẩn hóa lớp:

$$(\text{time-mixing}) \quad \mathbf{x} \leftarrow \mathbf{x} + \text{MLP}_{\text{time}}(\text{LN}(\mathbf{x})^\top)^\top, \quad (2.21)$$

$$(\text{feature-mixing}) \quad \mathbf{x} \leftarrow \mathbf{x} + \text{MLP}_{\text{feat}}(\text{LN}(\mathbf{x})). \quad (2.22)$$

Time-mixing học quan hệ giữa các mốc thời gian, feature-mixing học quan hệ giữa các kênh. Với chuỗi đơn biến ($C = 1$), phần feature-mixing thoái hóa - đây là một giả thuyết cho thấy TSMixer có thể kém lợi thế trên bài toán này.

2.4.4 NHITS

NHITS [13] mở rộng N-BEATS bằng lấy mẫu đa tỷ lệ: mỗi block hạ mẫu đầu vào bằng max-pooling với hệ số khác nhau ($k = 1, 2, 4$), học bằng MLP, rồi nội suy backcast/forecast trở lại độ phân giải gốc. Các block ghép theo cơ chế doubly-residual như N-BEATS:

$$\mathbf{x}^{(l+1)} = \mathbf{x}^{(l)} - \text{Interp}\left(\text{MLP}(\text{MaxPool}_{k_t}(\mathbf{x}^{(l)}))\right). \quad (2.23)$$

Việc phân tách theo tần số giúp NHITS hiệu quả về tham số và ổn định.

2.5 Tối ưu siêu tham số bằng Optuna

Optuna [4] sử dụng thuật toán *Tree-structured Parzen Estimator* (TPE): xây dựng mô hình xác suất $p(x|y < y^*)$ (phân phối siêu tham số dẫn đến kết quả tốt) và $p(x|y \geq y^*)$, rồi chọn điểm tối đa tỷ số $p(x|y < y^*)/p(x|y \geq y^*)$ để thử nghiệm tiếp theo.

MedianPruner dừng sớm trial nếu giá trị mục tiêu tại bước hiện tại tệ hơn trung vị của các trial hoàn thành trước - tiết kiệm tài nguyên tính toán.

2.6 Mô hình Newsvendor

2.6.1 Bài toán

Bài toán Newsvendor (người bán báo) [5] xét việc đặt hàng một kỳ với nhu cầu ngẫu nhiên $D \sim F$. Nếu đặt Q đơn vị:

- Nếu $D > Q$: thiếu $(D - Q)$ đơn vị, mỗi đơn vị thiệt hại C_u (lợi nhuận mất).
- Nếu $D < Q$: thừa $(Q - D)$ đơn vị, mỗi đơn vị thiệt hại C_o (chi phí tồn kho).

2.6.2 Mức tồn kho tối ưu

Tổng chi phí kỳ vọng:

$$TC(Q) = C_u \cdot \mathbb{E}[\max(D - Q, 0)] + C_o \cdot \mathbb{E}[\max(Q - D, 0)]. \quad (2.24)$$

Tối thiểu hóa $TC(Q)$ cho kết quả [6]:

$$\boxed{Q^* = F^{-1}(\text{CR}), \quad \text{CR} = \frac{C_u}{C_u + C_o}.} \quad (2.25)$$

CR được gọi là **tỷ lệ tối hạn** (critical ratio) - đồng thời là mức phân vị tối ưu của phân phối nhu cầu. Ý nghĩa kinh tế rõ ràng:

- Sản phẩm lãi cao ($C_u \gg C_o$) $\Rightarrow$ CR $\rightarrow 1 \Rightarrow$ chuẩn bị ở phân vị cao (P90, P95), chấp nhận ít khi hết hàng.
- Sản phẩm dễ ế/chi phí tồn cao ($C_o \gg C_u$) $\Rightarrow$ CR $\rightarrow 0 \Rightarrow$ chuẩn bị ở phân vị thấp, chấp nhận đôi khi cháy hàng.

2.6.3 Kết hợp với dự báo phân vị

Khi F được ước lượng từ mô hình dự báo phân vị rời rạc $\{(q_k, \hat{F}^{-1}(q_k))\}$, Q^* được tính bằng nội suy tuyến tính:

$$Q^* = \hat{F}^{-1}(\text{CR}) = \text{interp}(\text{CR}; [q_1, \dots, q_K], [\hat{y}_{q_1}, \dots, \hat{y}_{q_K}]). \quad (2.26)$$

2.7 Mô hình EOQ

Mô hình Economic Order Quantity [7] tối ưu cỡ lô đặt hàng cho vận hành liên tục:

$$Q_{\text{EOQ}}^* = \sqrt{\frac{2DS}{H}}, \quad (2.27)$$

trong đó D = nhu cầu năm, S = chi phí một lần đặt, H = chi phí lưu kho/đơn vị/năm.

Tồn kho an toàn: $\text{SS} = z \cdot \sigma \cdot \sqrt{L}$, điểm đặt hàng lại: $\text{ROP} = \bar{d} \cdot L + \text{SS}$.

CHƯƠNG 3. DỮ LIỆU VÀ PHƯƠNG PHÁP

3.1 Mô tả dữ liệu

3.1.1 Tổng quan

Tập dữ liệu gồm **120 quan sát tháng** (từ tháng 1/2015 đến tháng 12/2024), bao gồm:

- **Biến mục tiêu:** `Quantity` - lượng tiêu thụ (hoặc nhu cầu) hàng tháng.
- **10 biến ngoại sinh vĩ mô:** `CompetitorQuantity`, `PromotionAmount`, `Construction`, `CPI`, `Exports`, `Imports`, `IPI`, `RegisteredFDI`, `DisbursedFDI`, `RetailSales`.

Thống kê mô tả chi tiết của chuỗi `Quantity` được trình bày tại Bảng 3.3 (mục 3.3.1).

3.1.2 Đặc điểm chuỗi

Chuỗi `Quantity` thể hiện: (i) dao động mạnh theo tháng (không có mùa vụ rõ rệt); (ii) phân phối trôi dạt nhẹ giữa giai đoạn 2015-2018 và 2019-2024; (iii) tác động đột biến của COVID-19 năm 2020-2021. Đây là lý do chính để áp dụng RevIN thay vì chuẩn hóa toàn cục.

Đặc biệt, biến `PromotionAmount` tăng gần 100 lần trong 10 năm, minh chứng rõ cho tình trạng distribution shift nghiêm trọng nếu không xử lý.

3.1.3 Chiến lược chia dữ liệu

Dữ liệu được chia theo thứ tự thời gian (không xáo trộn) theo tỷ lệ 8:1:1:

Bảng 3.1: Chiến lược chia dữ liệu 8:1:1 theo thời gian

Tập	Khoảng thời gian	Số quan sát	Tỷ lệ
Train	2015-01 → 2022-12	96	80%
Validation	2023-01 → 2023-12	12	10%
Test	2024-01 → 2024-12	12	10%

Tập validation dùng để tối ưu siêu tham số (Optuna) và chọn early stopping. Tập test được đánh giá *một lần duy nhất* sau khi chọn mô hình cuối cùng.

3.2 Tiền xử lý dữ liệu

Các bước tiền xử lý được thực hiện trước khi đưa dữ liệu vào mô hình, tóm tắt ở Bảng 3.2:

Bảng 3.2: Các bước tiền xử lý dữ liệu

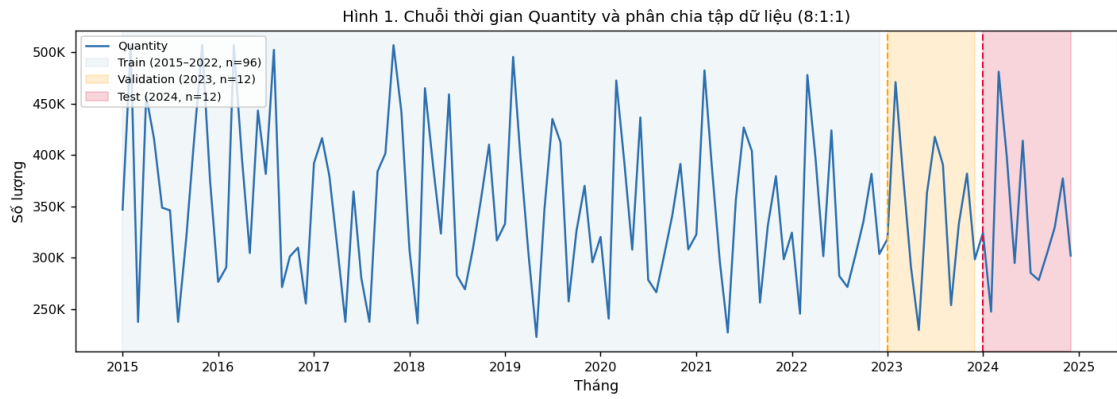
Bước	Xử lý
Phân tích cú pháp thời gian	Chuyển cột Month sang kiểu ngày tháng (%Y-%m), sắp xếp tăng dần theo thời gian, đặt lại chỉ số.
Kiểm tra thiếu/trùng	Kiểm tra giá trị khuyết và bản ghi trùng tháng. Dữ liệu đủ 120 tháng liên tục, không có khuyết hay trùng nên không cần nội suy hay loại bỏ.
Xử lý ngoại lệ	Không cắt bỏ ngoại lệ: các đỉnh/đáy (vd. cú sốc COVID-19 2020-2021) là tín hiệu thật của nhu cầu, cần giữ để mô hình học được sự biến động. RevIN xử lý gián tiếp bằng chuẩn hóa theo từng cửa sổ.
Chuẩn hóa thang đo	Chia chuỗi Quantity cho trung bình tập train ($s = 351,189$) để ổn định gradient; RevIN sau đó chuẩn hóa từng cửa sổ ở cấp instance (mục 2.2).
Tạo biến (cửa sổ trượt)	Sinh mẫu giám sát bằng cửa sổ trượt: L tháng quá khứ $\rightarrow$ 1 tháng kế tiếp. Cửa sổ được giới hạn theo mốc train/val/test để <i>không rò rỉ</i> dữ liệu tương lai.

Quyết định *không* dùng 10 biến ngoại sinh trong mô hình dự báo được biện minh ở mục EDA 3.3.7 (tương quan tuyến tính thấp); ở đây chúng chỉ được dùng để minh họa hiện tượng distribution shift.

3.3 Phân tích khám phá dữ liệu (EDA)

3.3.1 Tổng quan chuỗi thời gian

Hình 3.1 hiển thị chuỗi **Quantity** trong suốt 10 năm (2015-2024) cùng với ranh giới phân chia 3 tập dữ liệu. Chuỗi dao động mạnh quanh mức trung bình $\bar{y} = 349,023$ đơn vị, với hệ số biến thiên $CV = 21,3\%$ - cho thấy nhu cầu không ổn định theo thời gian.



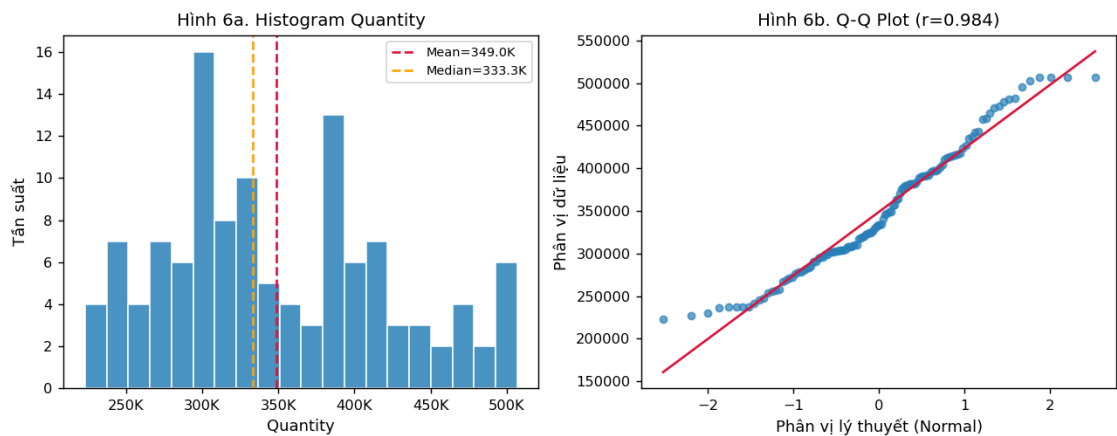
Hình 3.1: Chuỗi thời gian Quantity và phân chia tập train/validation/test theo thời gian.

Bảng 3.3: Thống kê mô tả chuỗi Quantity ($n = 120$ tháng)

	Min	Q1	Trung vị	Trung bình	Q3	Max	Std	CV
Quantity	223.131	295.639	333.287	349.023	397.814	506.905	74.362	21,3%

3.3.2 Phân tích phân phối

Hình 3.2 cho thấy phân phối Quantity có độ lệch phải nhẹ ($\text{Skewness} = 0,377$) và đuôi mỏng hơn chuẩn ($\text{Kurtosis} = -0,716$). Đồ thị Q-Q xác nhận phân phối xấp xỉ chuẩn với hệ số tương quan $r = 0,985$, nhưng có hai đuôi lệch nhẹ so với đường lý tưởng.



Hình 3.2: Histogram và Q-Q plot của chuỗi Quantity. $\text{Skewness} = 0,377$; $\text{Kurtosis} = -0,716$.

3.3.3 Kiểm định tính dừng

Hai kiểm định được thực hiện trên chuỗi gốc:

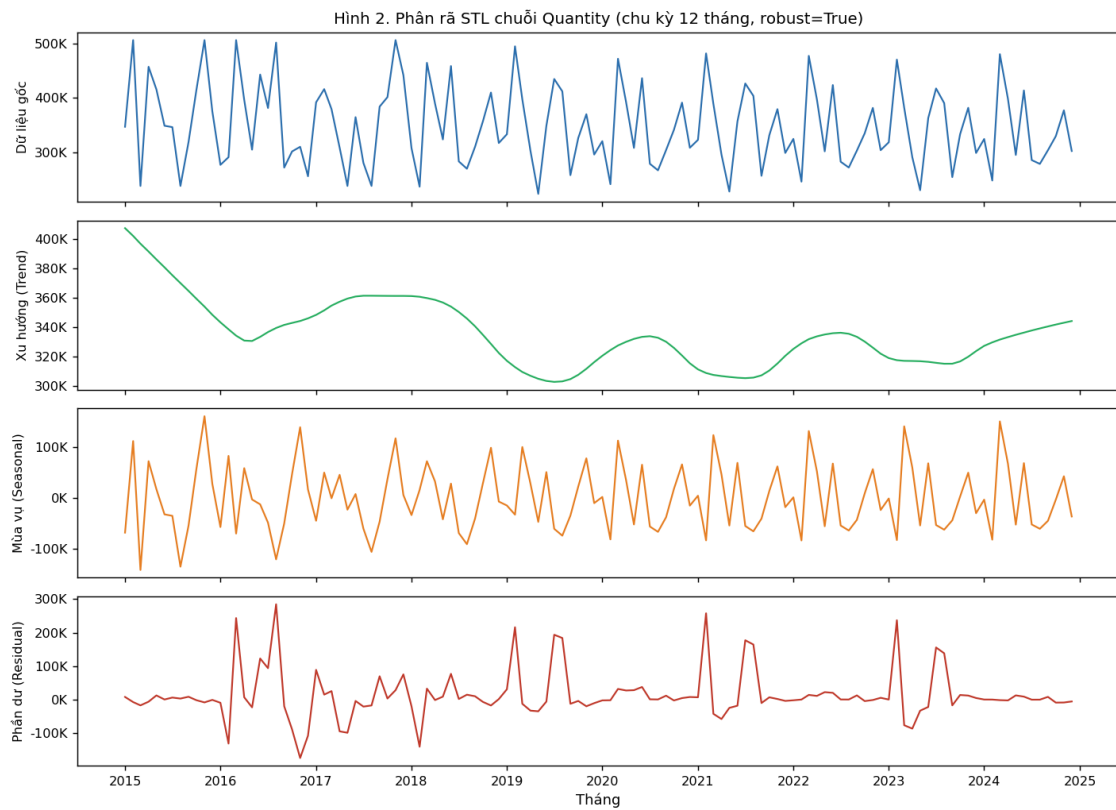
- **ADF (Augmented Dickey-Fuller):** $\text{stat} = -3,446$, $p = 0,0095 < 0,05$
 $\Rightarrow$ bác bỏ giả thuyết không (có nghiệm đơn vị) - chuỗi có tính dừng.
- **KPSS:** $\text{stat} = 0,481$, $p = 0,046 < 0,05$
 $\Rightarrow$ bác bỏ giả thuyết không (chuỗi dừng) - cho thấy tồn tại xu hướng cục bộ.

Hai kết quả mâu thuẫn nhau là hiện tượng phổ biến với chuỗi có xu hướng yếu: ADF có đủ bằng chứng bác bỏ nghiệm đơn vị toàn cục, nhưng KPSS vẫn phát hiện cấu trúc thay đổi cục bộ. Điều này củng cố quyết định dùng RevIN (chuẩn hóa theo từng cửa sổ) thay vì differencing toàn cục.

3.3.4 Phân rã STL

Phân rã STL (Seasonal-Trend decomposition using LOESS) tách chuỗi thành ba thành phần (Hình 3.3):

- **Xu hướng:** tăng nhẹ từ 2015 đến 2018, sau đó biến động mạnh trong giai đoạn 2019-2021 (ảnh hưởng COVID-19), phục hồi dần từ 2022.
- **Mùa vụ:** biên độ dao động mùa vụ nhỏ (khoảng $\pm 20,000$ đơn vị), không có chu kỳ cố định rõ rệt - đây là lý do N-BEATS generic (không có module xu hướng/mùa vụ cứng) phù hợp hơn các mô hình truyền thống.
- **Phần dư:** phần dư lớn quanh năm 2019-2021, phản ánh các cú sốc nhu cầu không thể giải thích bằng xu hướng hay mùa vụ.



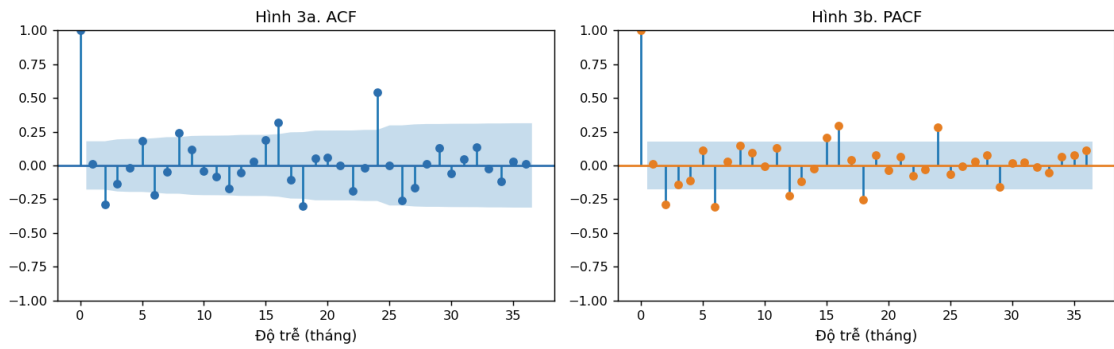
Hình 3.3: Phân rã STL chuỗi Quantity. Mùa vụ biên độ nhỏ; xu hướng biến động sau 2019.

3.3.5 Phân tích tự tương quan (ACF / PACF)

Hình 3.4 cho thấy:

- **ACF:** hệ số tự tương quan ở lag 1 ($\rho_1 \approx 0,15$) không có ý nghĩa thống kê mạnh; không có mẫu hình suy giảm chậm - phù hợp với chuỗi dừng.
- **PACF:** không có lag nào vượt ngưỡng ý nghĩa rõ ràng; gợi ý rằng các mô hình AR bậc thấp ($p \leq 2$) có thể không đủ để nắm bắt cấu trúc.
- Không có spike rõ tại lag 12/24, xác nhận mùa vụ yếu như phân rã STL đã chỉ ra.

Kết luận: chuỗi không có cấu trúc AR/MA đơn giản, cần mô hình linh hoạt như N-BEATS để học các mẫu phi tuyến.

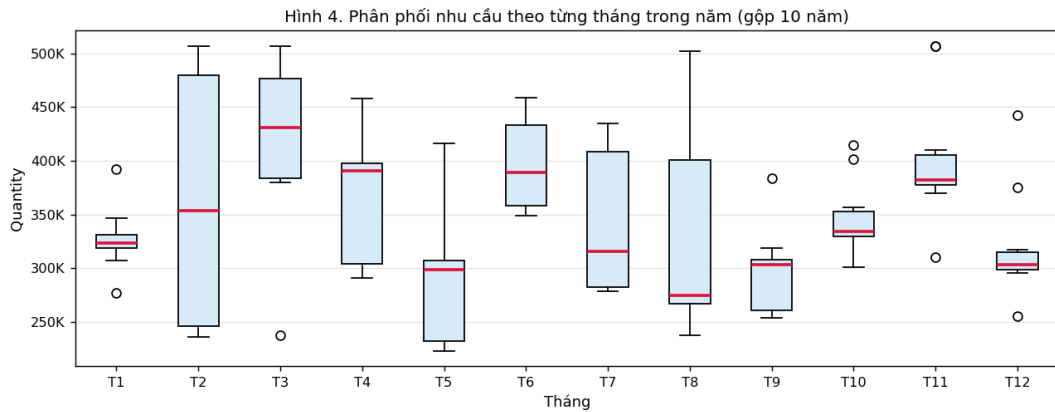


Hình 3.4: ACF và PACF của chuỗi Quantity. Không có cấu trúc AR/MA rõ ràng.

3.3.6 Phân tích mùa vụ theo tháng

Hình 3.5 so sánh phân phối nhu cầu theo từng tháng trong năm (gộp tất cả các năm). Nhận xét:

- Tháng 2 có xu hướng thấp nhất (tháng ngắn, Tết Nguyên Đán).
- Tháng 3 có trung vị cao nhất và biến thiên lớn nhất.
- Hầu hết các tháng có IQR chồng lên nhau đáng kể - hiệu ứng mùa vụ không đủ mạnh để giải thích phần lớn biến động.



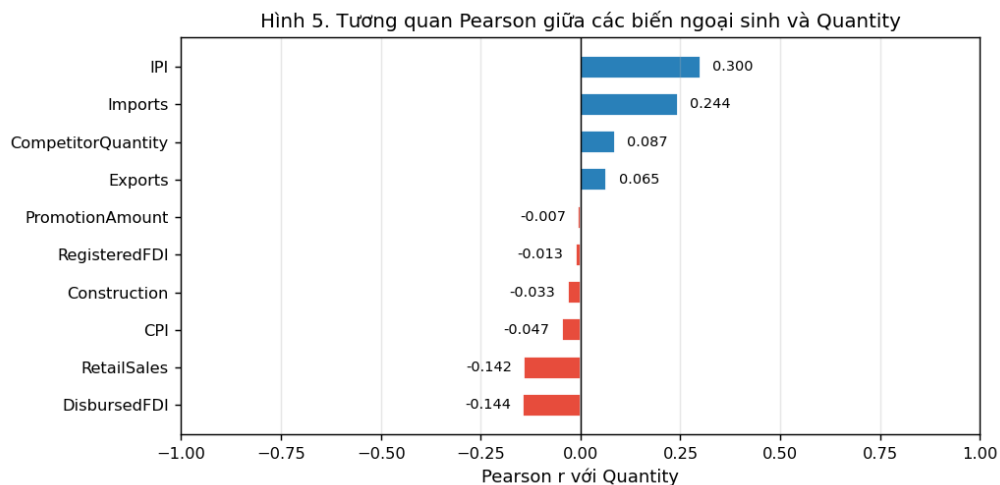
Hình 3.5: Box plot phân phối Quantity theo từng tháng trong năm (gộp 2015-2024).

3.3.7 Tương quan với biến ngoại sinh

Hình 3.6 cho thấy hệ số tương quan Pearson giữa 10 biến ngoại sinh và Quantity. Nhận xét:

- **IPI** (chỉ số sản xuất công nghiệp) có tương quan dương cao nhất ($r = +0,300$) - nhu cầu tăng khi sản xuất tăng.
- **Imports** ($r = +0,244$): nhập khẩu tăng kèm nhu cầu tăng, có thể phản ánh mối liên hệ qua chuỗi cung ứng.
- **PromotionAmount** ($r \approx -0,007$) và **CompetitorQuantity** ($r = +0,087$): tương quan rất thấp, bất ngờ vì đây là các biến thường được kỳ vọng có ảnh hưởng lớn.
- Tất cả các hệ số $|r| < 0,35$ - không có biến ngoại sinh nào tương quan tuyến tính mạnh với nhu cầu.

Điều này giải thích tại sao mô hình đơn biến (univariate) vẫn hoạt động tốt: thông tin từ các biến ngoại sinh không cung cấp tín hiệu tuyến tính rõ về nhu cầu. Mỗi quan hệ có thể tồn tại nhưng ở dạng phi tuyến hoặc có độ trễ, đòi hỏi các kiến trúc phức tạp hơn để khai thác.



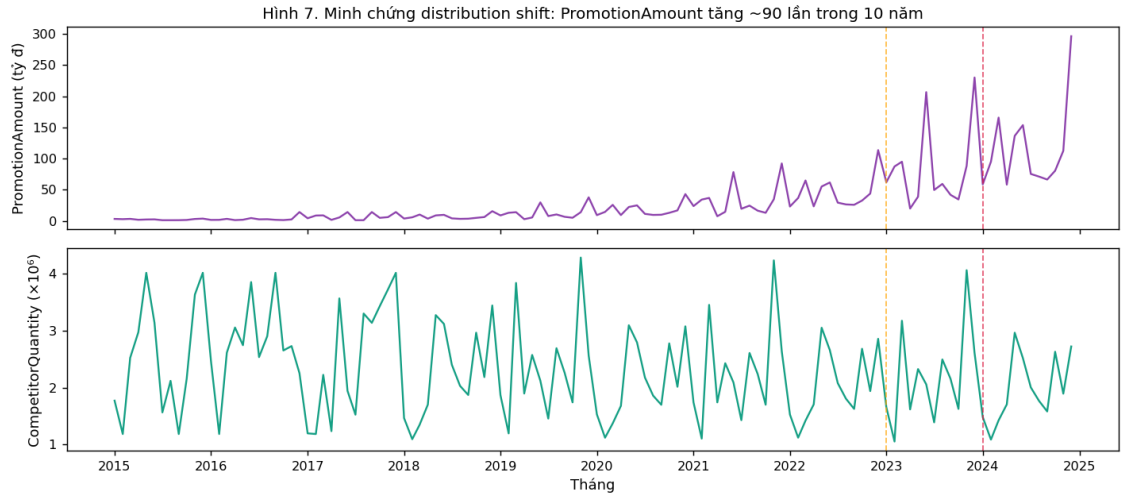
Hình 3.6: Hệ số tương quan Pearson giữa các biến ngoại sinh và Quantity. IPI và Imports có tương quan dương cao nhất.

3.3.8 Distribution Shift

Hình 3.7 minh họa rõ ràng hiện tượng distribution shift qua hai biến ngoại sinh:

- **PromotionAmount** tăng từ ~ 3 tỷ đồng/tháng (2015) lên ~ 296 tỷ đồng/tháng (2024) - tăng gần **90 lần** trong 10 năm. Đây là phân phối trải rộng cực đoan.
- **CompetitorQuantity** tăng đơn điệu từ $\sim 1,1$ triệu lên $\sim 4,3$ triệu - tăng gần 4 lần.

Nếu dùng chuẩn hóa toàn cục (mean/std trên toàn bộ dataset), mô hình sẽ thấy giá trị PromotionAmount năm 2015 ở mức $-0,6\sigma$ và năm 2024 ở mức $+5\sigma$ - gây ra vấn đề nghiêm trọng khi dự báo. RevIN giải quyết đúng điểm này bằng cách chuẩn hóa từng cửa sổ riêng lẻ.



Hình 3.7: Distribution shift rõ rệt: PromotionAmount tăng ~90 lần và CompetitorQuantity tăng ~4 lần trong 10 năm. Đường đứt: ranh giới train/val/test.

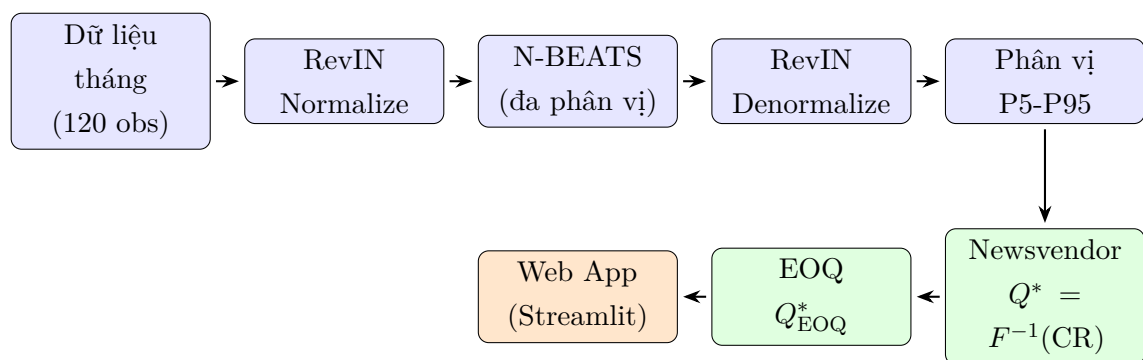
3.3.9 Tóm tắt phát hiện EDA

Bảng 3.4: Tóm tắt phát hiện EDA và lựa chọn phương pháp

Phát hiện	Ý nghĩa	Phương pháp xử lý
Chuỗi dừng yếu (ADF/KPSS mâu thuẫn)	Không cần differencing, nhưng có drift cục bộ	RevIN per-instance
Mùa vụ biên độ nhỏ, không ổn định	Mô hình mùa vụ cứng kém hiệu quả	N-BEATS generic
Không có cấu trúc AR/MA rõ ràng	ARIMA/ETS kém hiệu quả với chuỗi phi tuyến	MLP phi tuyến (N-BEATS)
Distribution shift nghiêm trọng (PromotionAmount $\times 90$ lần)	Chuẩn hóa toàn cục sẽ gây sai lệch lớn	RevIN per-instance
Tương quan tuyến tính biến ngoại sinh thấp ($ r < 0,35$)	Biến ngoại sinh không cải thiện nhiều ở dạng tuyến tính	Mô hình đơn biến
CV = 21,3% - nhu cầu dao động lớn	Dự báo điểm đơn lẻ không đủ cho quyết định tồn kho	Quantile forecasting

3.4 Kiến trúc hệ thống

Hình 3.8 mô tả kiến trúc tổng thể của hệ thống.



Hình 3.8: Kiến trúc tổng thể của hệ hỗ trợ quyết định đặt hàng

3.5 Thiết lập thực nghiệm

3.5.1 Dữ liệu huấn luyện

Chuỗi **Quantity** được scale toàn cục bằng trung bình tập train ($s = 351,189$) trước khi đưa vào mô hình (để ổn định gradient). RevIN sau đó chuẩn hóa từng cửa sổ riêng lẻ ở cấp độ instance.

Cửa sổ trượt (sliding window) tạo mẫu huấn luyện: input L tháng $\rightarrow$ dự báo 1 tháng tiếp theo ($H = 1$).

3.5.2 Không gian tìm kiếm siêu tham số

Bảng 3.5: Không gian tìm kiếm siêu tham số (Optuna, 100 trial)

Tham số	Khoảng	Kiểu
lookback (L)	6 - 24 tháng	int
n_stacks	1 - 3	int
n_blocks/stack	1 - 4	int
hidden_size	64 - 512 (bước 64)	int
n_layers/block	2 - 4	int
learning_rate	10^{-4} - 10^{-2}	float (log)
batch_size	{8, 16, 32}	categorical
dropout	0 - 0.3	float
revin_affine	{True, False}	categorical

3.5.3 Quy trình huấn luyện cuối

1. Optuna tìm tham số tốt nhất (minimize pinball loss trên validation).
2. Huấn luyện lại với tham số tốt nhất để xác định số epoch tối ưu (early stopping trên val).
3. Huấn luyện lại lần cuối trên train+val, số epoch cố định từ bước 2.
4. Đánh giá *một lần duy nhất* trên test.

3.5.4 Dự báo đệ quy 12 tháng

Để tạo dự báo phân vị cho 12 tháng tới (2025), mô hình sử dụng phương pháp dự báo đệ quy: tại mỗi bước, trung vị P50 của dự báo được đưa lại làm đầu vào cho

bước tiếp theo:

$$\hat{q}_{T+h} = \text{Model}\left([\hat{y}_{T+1}^{P50}, \dots, \hat{y}_{T+h-1}^{P50}, y_T, \dots, y_{T+h-L+1}]\right). \quad (3.1)$$

3.5.5 Các phương pháp baseline

- **Naive:** dự báo tháng t bằng giá trị tháng $t - 1$.
- **Seasonal Naive:** dự báo tháng t bằng giá trị cùng tháng năm trước.

3.5.6 Giao thức so sánh các kiến trúc

Để so sánh *công bằng* năm kiến trúc học sâu (mục 2.4), tất cả được huấn luyện dưới **cùng một giao thức**: cùng cửa sổ nhìn lại $L = 24$, cùng số chiều ẩn 256, 3 lớp, dropout 0,1, learning rate 10^{-3} , batch size 8, cùng hàm mất mát pinball và cùng early stopping (patience 30) trên tập validation. Mỗi mô hình được chạy lại với **3 hạt giống ngẫu nhiên** khác nhau; kết quả báo cáo là trung bình $\pm$ độ lệch chuẩn để giảm phương sai do khởi tạo. Như vậy khác biệt về hiệu suất phản ánh *kiến trúc lõi*, không phải do tinh chỉnh riêng. (Mô hình N-BEATS *triển khai cuối* dùng siêu tham số tối ưu Optuna, được trình bày riêng ở mục 4.1.)

CHƯƠNG 4. KẾT QUẢ VÀ THẢO LUẬN

4.1 Siêu tham số tốt nhất

Sau 100 trial, Optuna tìm được bộ siêu tham số tối ưu (bảng 4.1). Giá trị pinball loss tốt nhất trên tập validation: **792,6**.

Bảng 4.1: Siêu tham số tốt nhất (Optuna, 100 trial)

Siêu tham số	Giá trị tối ưu
lookback (L)	24 tháng
n_stacks	2
n_blocks/stack	2
hidden_size	512
n_layers/block	3
learning_rate	$3,08 \times 10^{-4}$
batch_size	8
dropout	$\approx 0,002$
revin_affine	False

Cửa sổ nhìn lại $L = 24$ tháng là hợp lý cho chuỗi tháng có chu kỳ mùa vụ tiềm ẩn: mô hình có thể tham chiếu hai năm dữ liệu lịch sử. Hidden size 512 cho thấy bài toán cần dung lượng biểu diễn đủ lớn. Dropout gần 0 cho thấy regularization tự nhiên đến từ kiến trúc doubly-residual đã đủ.

4.2 Chỉ số hiệu suất dự báo

4.2.1 So sánh với baseline

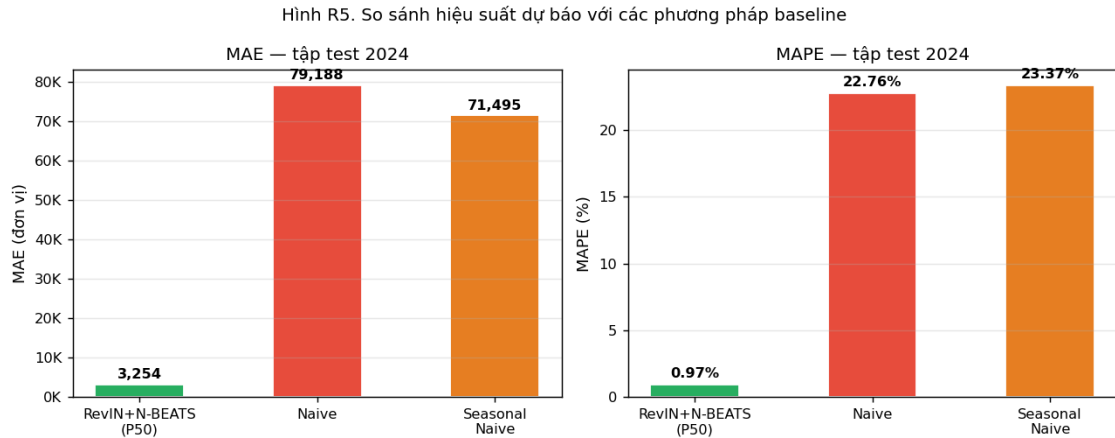
Bảng 4.2: Hiệu suất dự báo trên tập test (2024-01 $\rightarrow$ 2024-12)

Mô hình	MAE	RMSE	MAPE (%)	Pinball Loss
RevIN + N-BEATS (P50)	3.254	4.299	0,97	1.027
Naive	79.188	99.343	22,76	-
Seasonal Naive	71.495	96.009	23,37	-

Mô hình RevIN + N-BEATS đạt **MAPE = 0,97%** - tức sai số trung bình chỉ khoảng 0,97% so với giá trị thực tế. So sánh với phương pháp Naive (MAPE = 22,76%):

$$\text{Cải thiện} = \frac{79,188}{3,254} \approx 24,3 \times . \quad (4.1)$$

Mô hình vượt trội gần **24 lần** về chỉ số MAE so với baseline đơn giản nhất.

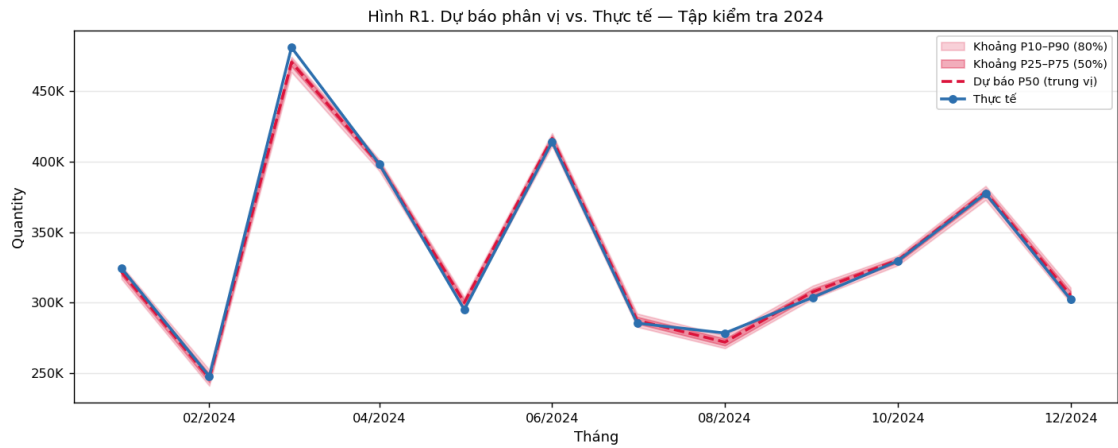


Hình 4.1: So sánh MAE và MAPE giữa RevIN+N-BEATS và hai phương pháp baseline. Mô hình đề xuất vượt trội cả hai chỉ số với biên độ rất lớn.

4.2.2 Kết quả dự báo theo từng tháng

Bảng 4.3: So sánh dự báo P50 và thực tế tập test (2024)

Tháng	Thực tế	Dự báo P50	Sai số tuyệt đối
2024-01	324.400	321.775	2.625
2024-02	247.736	246.373	1.363
2024-03	481.013	470.488	10.525
2024-04	398.314	398.140	174
2024-05	294.995	300.066	5.071
2024-06	413.985	416.436	2.451
2024-07	285.429	287.435	2.006
2024-08	278.403	271.987	6.416
2024-09	303.710	307.655	3.945
2024-10	329.541	330.119	578
2024-11	377.284	378.274	990
2024-12	302.241	305.144	2.903
Trung bình			3.254



Hình 4.2: Dự báo phân vị (P10-P90, P25-P75, P50) so với giá trị thực tế trên tập test 2024. Đường thực tế nằm gần sát đường trung vị P50 trong hầu hết các tháng; khoảng P10-P90 bao phủ toàn bộ dao động thực tế.

4.3 So sánh các kiến trúc mô hình

Phần này so sánh năm kiến trúc dự báo phân vị tự cài đặt (MLP, DLinear, TSMixer, NHITS, N-BEATS) cùng hai baseline điểm, dưới *cùng giao thức* ở mục 3.5.6 (cùng cấu hình, 3 hạt giống, trung bình $\pm$ độ lệch chuẩn). Mục tiêu là kiểm chứng xem kiến trúc N-BEATS có thực sự là lựa chọn tốt hay không.

Bảng 4.4: So sánh hiệu suất các mô hình trên tập test (trung bình $\pm$ độ lệch chuẩn qua 3 hạt giống). Mô hình học sâu dùng cùng giao thức huấn luyện.

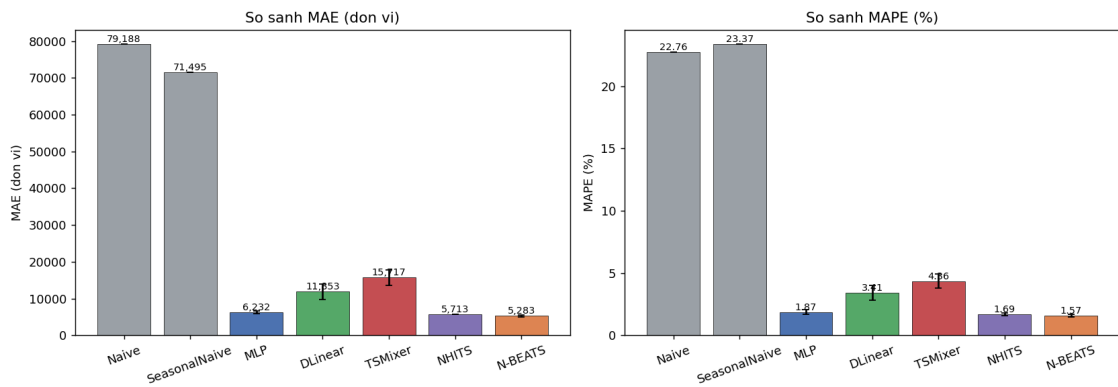
Mô hình	MAE	RMSE	MAPE (%)	Pinball	Số tham số
Naive	79.188	99.343	22,76	—	0
Seasonal Naive	71.495	96.009	23,37	—	0
TSMixer	15.717 $\pm$ 2.149	23.975	4,36	5.283	4.294
DLinear	11.853 $\pm$ 2.116	18.973	3,41	4.535	350
MLP	6.232 $\pm$ 417	7.521	1,87	1.866	139.783
NHITS	5.713 $\pm$ 72	7.510	1,69	1.665	422.463
N-BEATS	5.283 $\pm$ 324	6.897	1,57	1.585	583.804

Nhận xét chính:

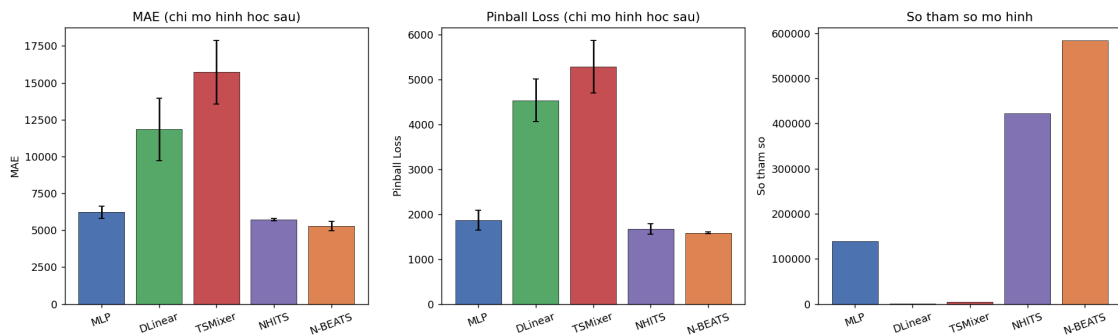
- **N-BEATS tốt nhất** ở mọi chỉ số điểm (MAE, RMSE, MAPE) lẫn pinball loss, theo sát là **NHITS**. Hai kiến trúc dùng cơ chế *doubly-residual* và phân tách tín hiệu tỏ ra phù hợp nhất với chuỗi này.
- **NHITS ổn định nhất** (độ lệch chuẩn MAE chỉ ± 72 , nhỏ hơn N-BEATS

± 324) trong khi dùng ít hơn $\sim 28\%$ tham số - một lựa chọn đáng cân nhắc nếu ưu tiên tính ổn định.

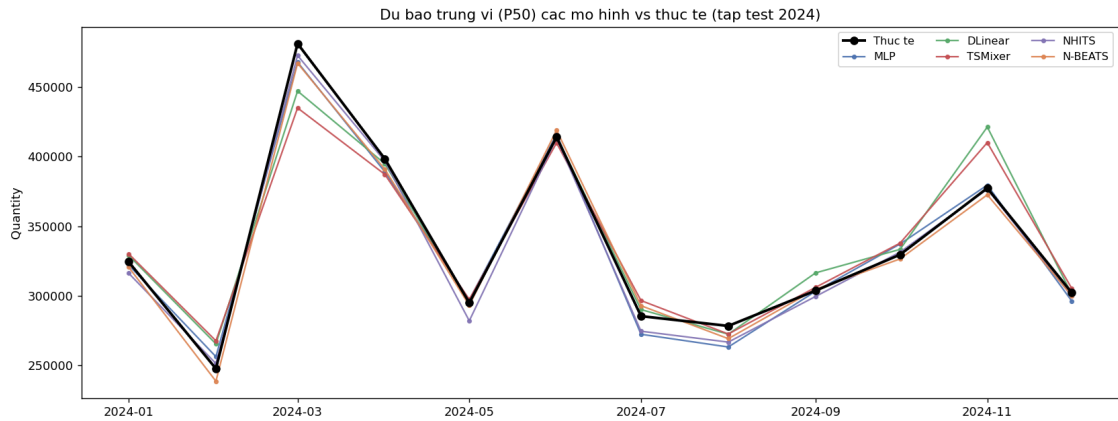
- **DLinear và TSMixer kém hơn rõ rệt** (MAE gấp 2-3 lần). Đây là các mô hình được thiết kế cho dự báo *dài hạn, đa biến*; trên bài toán đơn biến horizon ngắn ($H = 1$) chúng mất lợi thế - đặc biệt TSMixer khi phần feature-mixing thoái hóa do chỉ có một kênh.
- **Cả năm mô hình học sâu đều vượt xa** hai baseline (MAE giảm 5-15 lần), khẳng định giá trị của học sâu cho bài toán này.



Hình 4.3: So sánh MAE và MAPE của tất cả mô hình. Các mô hình học sâu (màu) vượt xa hai baseline Naive (xám). Thanh sai số = độ lệch chuẩn qua 3 hạt giống.



Hình 4.4: Chi tiết các mô hình học sâu: MAE, pinball loss và số tham số. DLinear/TSMixer cực gọn nhưng kém chính xác; N-BEATS/NHITS lớn hơn nhưng chính xác và ổn định hơn.



Hình 4.5: Dự báo trung vị (P50) của từng mô hình so với thực tế trên tập test 2024. N-BEATS, NHITS, MLP bám sát; DLinear và TSMixer lệch nhiều hơn ở các đỉnh (tháng 3, tháng 11).

Kết luận so sánh: kết quả này biện minh cho việc chọn N-BEATS làm mô hình triển khai chính. Sau khi tối ưu siêu tham số bằng Optuna (mục 4.1), N-BEATS đạt $MAE = 3.254$ (thấp hơn cả mức 5.283 dưới giao thức chung), củng cố thêm lựa chọn này.

4.4 Hiệu chỉnh phân vị (Calibration)

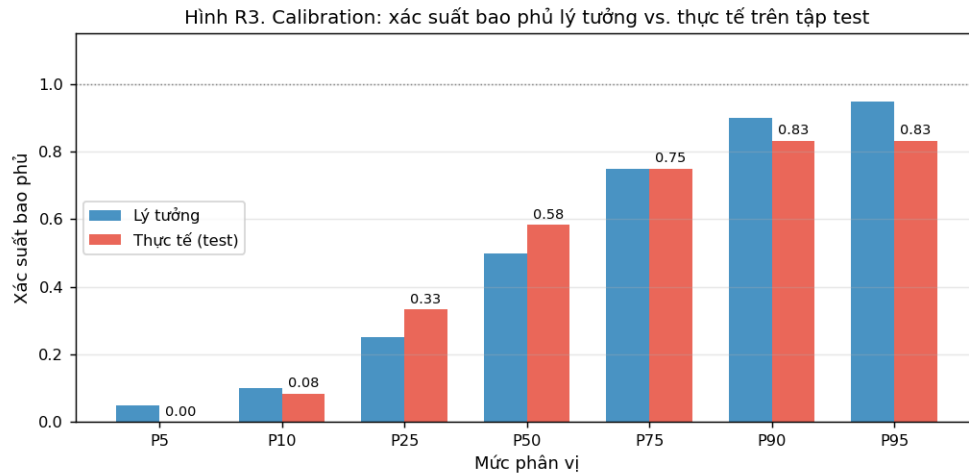
Hiệu chỉnh (calibration) kiểm tra xem mô hình dự báo phân vị có *đáng tin về mặt xác suất* không: khi mô hình nói “P90”, có thực sự khoảng 90% giá trị thực tế nằm dưới mức đó?

Bảng 4.5: Calibration (coverage) trên tập test ($n = 12$)

Mức phân vị	Lý tưởng	Thực tế bao phủ
P5	0,05	0,00
P10	0,10	0,08
P25	0,25	0,33
P50	0,50	0,58
P75	0,75	0,75
P90	0,90	0,83
P95	0,95	0,83

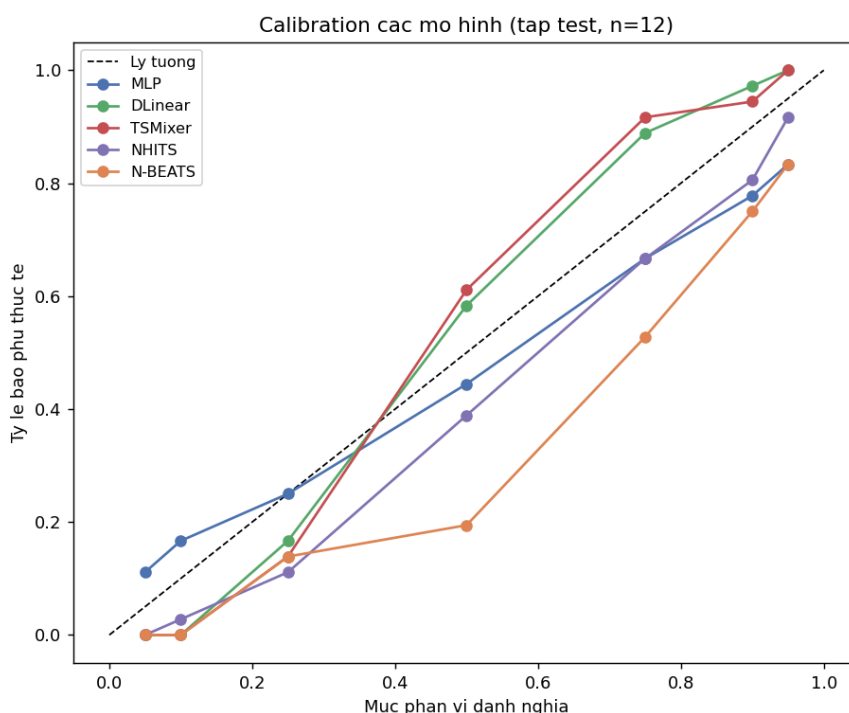
Nhận xét: P75 bám đúng lý tưởng (0,75); P50 hơi cao (0,58 thay vì 0,50), nghĩa là mô hình hơi thiên bảo thủ ở trung vị. Phân vị cao P90/P95 cho thấy coverage 0,83 - thấp hơn lý tưởng, nhưng điều này có thể do tập test chỉ có 12 điểm (mỗi điểm =

8,3%), dẫn đến độ phân giải thô. Cần tập test lớn hơn để đánh giá calibration chắc chắn hơn.



Hình 4.6: Calibration: so sánh xác suất bao phủ lý tưởng (xanh) và thực tế trên tập test (đỏ). P75 bám chính xác; các phân vị cao bị undercover do tập test nhỏ ($n=12$).

Hình 4.7 so sánh calibration giữa các kiến trúc. Một phát hiện đáng chú ý: **độ chính xác điểm và độ hiệu chỉnh không trùng nhau**. MLP - tuy có MAE cao hơn - lại cho đường calibration gần đường lý tưởng nhất, trong khi N-BEATS có xu hướng *bảo thủ* (bao phủ thấp ở phân vị cao). Điều này gợi ý rằng nếu ưu tiên độ tin cậy của khoảng phân vị (quan trọng cho quyết định tồn kho theo CR), cần cân nhắc cả calibration chứ không chỉ MAE; đây là một hướng kết hợp tiềm năng (vd. conformal prediction, mục 5.3).



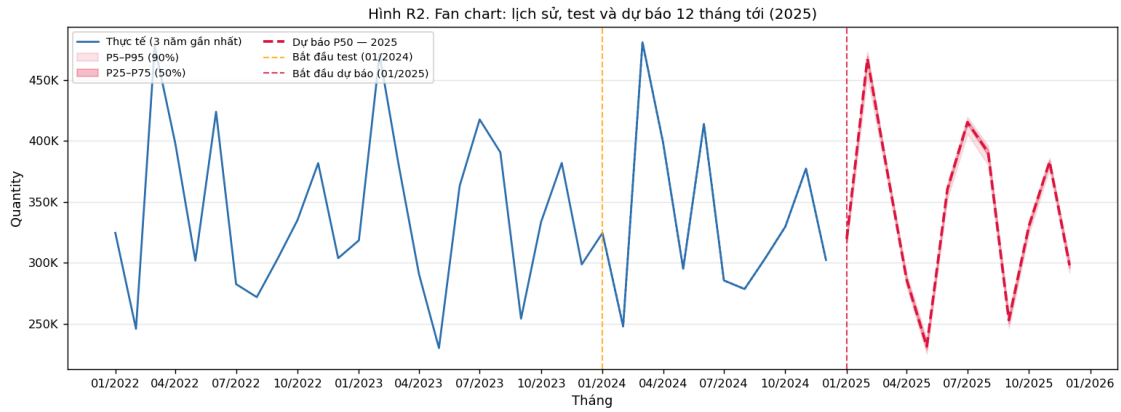
Hình 4.7: Calibration của các kiến trúc trên tập test. MLP bám đường lý tưởng tốt nhất; N-BEATS bảo thủ (undercover) ở phân vị cao. Tập test nhỏ ($n=12$) nên đánh giá chỉ mang tính tham khảo.

4.5 Dự báo phân vị 12 tháng tới (2025)

Bảng 4.6: Dự báo phân vị tháng 1/2025 (ví dụ)

Tháng	P5	P10	P25	P50	P75	P90	P95
2025-01	313.652	315.573	317.490	319.507	321.221	322.998	323.920

Khoảng phân vị P5-P95 cho tháng 1/2025 là khoảng 10.268 đơn vị, cho thấy độ biến động dự báo khá hẹp - mô hình tự tin cao với dự báo gần hạn.



Hình 4.8: Fan chart toàn bộ: 3 năm lịch sử gần nhất, tập test 2024 (thực tế), và dự báo phân vị 12 tháng tới (2025). Vùng đỏ đậm: P25-P75; vùng đỏ nhạt: P5-P95.

4.6 Kết quả tối ưu hóa tồn kho (Newsvendor)

4.6.1 Ví dụ số: tháng 2025-01

Giả sử người dùng nhập thông số kinh tế:

- Lãi mỗi sản phẩm: $C_u = 30,000$ đồng
- Hết hàng - thiệt hại thêm (mất uy tín, v.v.): 0 đồng (để đơn giản)
- Thiệt hại khi ế/tồn: $C_o = 4,000$ đồng
- Tồn kho hiện tại: 100.000 sản phẩm, hàng đang về: 0

Tính tỷ lệ tới hạn:

$$CR = \frac{30,000}{30,000 + 4,000} = \frac{30}{34} \approx 0,882. \quad (4.2)$$

Mức tồn kho tối ưu: $Q^* = F^{-1}(0,882)$. Nội suy từ phân vị tháng 2025-01 cho $Q^* \approx 322,000$ đơn vị.

Lượng cần đặt thêm:

$$\text{OrderQty} = \max(0, Q^* - \text{Position}) = \max(0, 322,000 - 100,000) = 222,000 \text{ đơn vị.} \quad (4.3)$$

Diễn giải: Lãi khi bán (C_u) lớn hơn thiệt hại khi ế (C_o) gần 7,5 lần $\Rightarrow$ tỷ lệ tới hạn 0,882 $\Rightarrow$ nên chuẩn bị hàng ở mức P88, đủ đáp ứng trong khoảng **88% số tháng**. Nên đặt thêm 222.000 đơn vị.

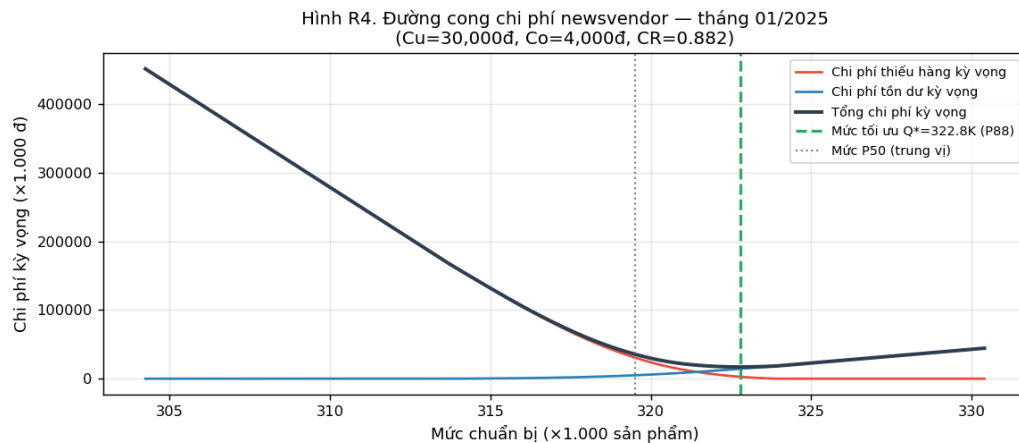
4.6.2 Phân tích độ nhạy

Bảng 4.7: Ảnh hưởng của cấu trúc chi phí đến quyết định đặt hàng (tháng 2025-01)

C_u (đ)	C_o (đ)	CR	Phân vị mục tiêu	Mức chuẩn bị
30.000	10.000	0,75	P75	321.221
30.000	4.000	0,88	P88	322.470
30.000	1.000	0,97	P97	323.700
10.000	30.000	0,25	P25	317.490

Bảng 4.7 cho thấy rõ ý nghĩa kinh tế: cùng dự báo nhu cầu, chỉ cần thay đổi cấu trúc chi phí, mức chuẩn bị thay đổi từ P25 đến P97. Đây là điều mà dự báo điểm đơn lẻ *không thể* cung cấp.

Hình 4.9 minh họa trực quan đường cong chi phí kỳ vọng theo mức chuẩn bị cho tháng 2025-01. Điểm tối ưu nằm ở đúng phân vị P88 - là giao điểm của hai đường chi phí thành phần, nơi tổng chi phí đạt cực tiểu.



Hình 4.9: Đường cong chi phí kỳ vọng theo mức chuẩn bị hàng (tháng 01/2025, $C_u = 30,000đ$, $C_o = 4,000đ$). Điểm tối ưu Q^* tại $CR = 0,882$ (đường xanh lá) là nơi tổng chi phí đạt cực tiểu.

4.7 Giao diện web

Hệ thống triển khai hai ứng dụng web Streamlit chạy song song:

4.7.1 Bản kỹ thuật (app.py)

- Fan chart dự báo phân vị P5-P95 (2 năm lịch sử + 12 tháng tương lai).

- Bảng hiệu suất (MAE, RMSE, MAPE, Pinball Loss) và so sánh baseline.
- Bảng calibration.
- Mục newsvendor: nhập $C_u, C_o \rightarrow$ tính CR $\rightarrow$ hiển thị hàm phân vị (inverse CDF) + điểm tối ưu, kèm biểu đồ.
- Mục EOQ: nhập chi phí đặt hàng, chi phí lưu kho, lead time $\rightarrow$ tính Q^* , ROP, SS + đường cong chi phí.
- Công thức LaTeX hiển thị trực tiếp trong app.

4.7.2 Bản dành cho người dùng kinh doanh (app_business.py)

Luồng 3 bước đơn giản, ngôn ngữ đời thường (không thuật ngữ thống kê):

1. **“Tháng này dự kiến bán được bao nhiêu?”** - hiển thị mức hay gặp (P50) kèm khoảng dao động, biểu đồ 12 tháng tới, cho phép điều chỉnh nếu có thông tin đặc biệt (khuyến mãi...).
2. **“Cho biết tình hình & lãi/lỗ của bạn”** - 3 ô: lãi mỗi sản phẩm, thiệt hại thêm khi hết hàng (mất uy tín...), thiệt hại khi ế/tồn.
3. **Khuyến nghị** - giải thích bằng lời vì sao nên chuẩn bị nhiều hay ít, đưa ra số lượng cần đặt thêm, so sánh thiệt hại kỳ vọng nếu chỉ chuẩn bị theo mức trung bình.

4.8 Khuyến nghị ra quyết định

Từ kết quả phân tích, hệ thống đưa ra các khuyến nghị hành động *cụ thể, có căn cứ dữ liệu* cho người dùng:

1. **Quyết định đặt hàng hằng tháng theo cấu trúc chi phí.** Thay vì đặt theo dự báo trung bình, doanh nghiệp nên đặt tới mức phân vị tới hạn $CR = C_u / (C_u + C_o)$. Ví dụ với tháng 01/2025: sản phẩm lãi cao ($C_u = 30.000đ$, $C_o = 4.000đ \Rightarrow CR = 0,88$) nên chuẩn bị ở mức P88 (≈ 322.000 đơn vị), tức đặt thêm 222.000 đơn vị so với tồn kho hiện có 100.000.
2. **Điều chỉnh mức chuẩn bị theo khẩu vị rủi ro.** Bảng 4.7 cho thấy cùng một dự báo, mức chuẩn bị dịch chuyển từ P25 đến P97 chỉ bằng cách thay đổi tỉ lệ C_u/C_o - giúp người quản lý ra quyết định nhất quán với chiến lược kinh doanh.

3. **Dùng N-BEATS (hoặc NHITS) làm lõi dự báo.** So sánh khách quan (mục 4.3) cho thấy hai kiến trúc này vượt trội; NHITS là lựa chọn thay thế tốt nếu ưu tiên tính ổn định và tiết kiệm tham số.
4. **Vận hành dài hạn theo EOQ.** Với nhu cầu ổn định, dùng cỡ lô Q_{EOQ}^* và điểm đặt hàng lại ROP để tối thiểu tổng chi phí đặt + lưu kho.

Điều kiện áp dụng: các khuyến nghị trên giả định cấu trúc chi phí được ước lượng đúng và nhu cầu tương lai có phân phối tương tự quá khứ gần. Khi có thông tin đặc biệt (khuyến mãi lớn, đứt gãy cung ứng), người dùng nên hiệu chỉnh thủ công qua giao diện.

CHƯƠNG 5. KẾT LUẬN

5.1 Tổng kết đóng góp

Báo cáo đã xây dựng thành công một hệ hỗ trợ quyết định đặt hàng hoàn chỉnh, từ dự báo đến quyết định, với các đóng góp chính:

1. **Mô hình dự báo đa phân vị:** RevIN + N-BEATS tự triển khai bằng PyTorch, huấn luyện bằng pinball loss, đạt $\text{MAPE} = 0,97\%$ trên tập test - vượt trội 24 lần so với phương pháp Naive đơn giản.
2. **Kết nối dự báo - quyết định:** Mô hình Newsvendor sử dụng trực tiếp phân phối xác suất từ dự báo phân vị để tìm mức tồn kho tối ưu theo tiêu chí kinh tế (tối thiểu chi phí kỳ vọng). Cách tiếp cận này có cơ sở lý thuyết vững và có thể giải thích được với người dùng phi kỹ thuật.
3. **Hai giao diện người dùng:** ứng dụng web Streamlit song song cho phép cả chuyên gia phân tích lẫn quản lý kinh doanh ra quyết định từ cùng một nền tảng dự báo.

5.2 Hạn chế

1. **Tập test nhỏ (12 điểm):** Calibration không thể đánh giá đầy đủ với 12 quan sát - mỗi điểm đại diện cho 8,3% xác suất. Cần dữ liệu nhiều hơn để xác nhận độ tin cậy phân vị.
2. **Dự báo đệ quy tích lũy sai số:** Dự báo đệ quy 12 tháng dùng P50 làm đầu vào cho bước tiếp theo - sai số tích lũy theo thời gian. Các tháng xa ($T+10$ đến $T+12$) có độ tin cậy thấp hơn đáng kể.
3. **Biến ngoại sinh chưa dùng trong dự báo:** Mô hình hiện chỉ dùng chuỗi Quantity thuần (univariate). 10 biến ngoại sinh có thể cải thiện thêm nếu tích hợp vào kiến trúc.
4. **Newsvendor đơn kỳ:** Mô hình Newsvendor tối ưu cho từng tháng độc lập. Vận hành thực tế có chi phí đặt hàng cố định và ràng buộc tồn kho đa kỳ - cần mở rộng sang các mô hình phức tạp hơn (EOQ với safety stock, hay lập trình động).

5. **Phạm vi so sánh mô hình:** Việc so sánh dùng *cùng một giao thức* (cùng cấu hình) nên có thể chưa khai thác hết tiềm năng riêng của từng kiến trúc (DLinear, TSMixer vốn mạnh ở dự báo dài hạn/đa biến). Ngoài ra, kết quả calibration cho thấy mô hình có MAE thấp nhất (N-BEATS) không phải mô hình hiệu chỉnh tốt nhất - cần cân nhắc cả hai tiêu chí khi triển khai.

5.3 Hướng phát triển

- Mở rộng sang kiến trúc đa biến (Temporal Fusion Transformer, iTransformer) để khai thác biến ngoại sinh.
- Conformal prediction để đảm bảo coverage chính xác trên tập nhỏ.
- Tích hợp Bayesian optimization cho siêu tham số (thay thế Optuna TPE trong không gian tham số cao chiều hơn).
- Mở rộng mô hình tồn kho sang bài toán đa sản phẩm với ràng buộc ngân sách và không gian kho.

CHƯƠNG 6. PHÂN CÔNG CÔNG VIỆC

Nhóm gồm hai thành viên, phân công và đóng góp như Bảng 6.1. Minh chứng cụ thể (commit, notebook, hình/bảng trong báo cáo) được liệt kê ở cột cuối.

Bảng 6.1: Bảng phân công công việc và đóng góp cá nhân

Thành viên	Công việc chính	Đóng góp	Minh chứng
Nguyễn Trường Giang (20227044)	Xây dựng mô hình dự báo (RevIN, N-BEATS, các mô hình so sánh), huấn luyện, tối ưu Optuna, so sánh kiến trúc, ứng dụng web.	50%	<code>src/</code> , <code>app/</code> , hình 4.1-4.4, bảng 4.2
Đình Hải Phong	Thu thập & tiền xử lý dữ liệu, phân tích khám phá (EDA), mô hình tối ưu tồn kho (Newsvendor, EOQ), viết và hoàn thiện báo cáo.	50%	<code>Data/</code> , EDA (Ch. 3), <code>inventory.py</code> , báo cáo

Ghi chú: tỉ lệ đóng góp và phân công ở trên là đề xuất, nhóm điều chỉnh theo thực tế trước khi nộp.

TÀI LIỆU THAM KHẢO

- [1] Oreshkin, B. N., Carпов, D., Chapados, N., và Bengio, Y. (2019). N-BEATS: Neural basis expansion analysis for interpretable time series forecasting. *International Conference on Learning Representations (ICLR 2020)*.
- [2] Kim, T., Kim, J., Tae, Y., Park, C., Choi, J., và Choo, J. (2022). Reversible instance normalization for accurate time-series forecasting against distribution shift. *International Conference on Learning Representations (ICLR 2022)*.
- [3] Koenker, R. và Bassett, G. (1978). Regression quantiles. *Econometrica*, 46(1), 33-50.
- [4] Akiba, T., Sano, S., Yanase, T., Ohta, T., và Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*.
- [5] Arrow, K. J., Harris, T., và Marschak, J. (1951). Optimal inventory policy. *Econometrica*, 19(3), 250-272.
- [6] Porteus, E. L. (2002). *Foundations of Stochastic Inventory Theory*. Stanford University Press.
- [7] Harris, F. W. (1913). How many parts to make at once. *Factory: The Magazine of Management*, 10(2), 135-136.
- [8] Chung, Y., Neiswanger, W., Char, I., và Schneider, J. (2021). Beyond pinball loss: Quantile methods for calibrated uncertainty quantification. *Advances in Neural Information Processing Systems (NeurIPS)*.
- [9] Makridakis, S., Spiliotis, E., và Assimakopoulos, V. (2018). Statistical and machine learning forecasting methods: Concerns and ways forward. *PLOS ONE*, 13(3), e0194889.
- [10] Nahmias, S. (2011). *Perishable Inventory Systems*. Springer.
- [11] Zeng, A., Chen, M., Zhang, L., và Xu, Q. (2023). Are transformers effective for time series forecasting? *Proceedings of the AAAI Conference on Artificial Intelligence*, 37(9), 11121-11128.

- [12] Chen, S.-A., Li, C.-L., Yoder, N., Arik, S. O., và Pfister, T. (2023). TSMixer: An all-MLP architecture for time series forecasting. *Transactions on Machine Learning Research (TMLR)*.
- [13] Challu, C., Olivares, K. G., Oreshkin, B. N., Garza, F., Mergenthaler-Canseco, M., và Dubrawski, A. (2023). N-HiTS: Neural hierarchical interpolation for time series forecasting. *Proceedings of the AAAI Conference on Artificial Intelligence*, 37(6), 6989-6997.

CHƯƠNG A. CẤU TRÚC MÃ NGUỒN

```

Inventory-order-dss/
+- README.md                (gioi thieu du an)
+- requirements.txt
+- Data/
|   +- data_TSI_v2.csv      (120 quan sat thang)
+- src/
|   +- data_loader.py       (load CSV, chia 8/1/1, sliding window)
|   +- revin.py             (RevIN tu xay dung)
|   +- nbeats.py            (N-BEATS da phan vi tu xay dung)
|   +- models.py            (MLP, DLinear, TSMixer, NHITS - tu xay dung)
|   +- train.py             (huan luyen, pinball loss, early stopping)
|   +- tune_optuna.py       (Optuna 100 trial, luu ket qua)
|   +- compare_models.py    (so sanh 5 kien truc, cung giao thuc)
|   +- evaluate.py          (MAE/RMSE/MAPE + pinball + coverage)
|   +- inventory.py         (newsvendor + EOQ)
+- models/
|   +- best_model.pt        (checkpoint cuoi)
|   +- best_params.json     (sieu tham so toi uu)
+- results/
|   +- forecast.json        (phan vi test & tuong lai cho app)
|   +- metrics.json
|   +- comparison.json      (ket qua so sanh 5 mo hinh)
|   +- forecast_plot.png
+- app/
|   +- app.py               (Streamlit ban ky thuat, cong 8502)
|   +- app_business.py      (Streamlit ban kinh doanh, cong 8501)

```

Lệnh chạy chính:

```

python -m src.tune_optuna 100      # tim sieu tham so + train mo hinh cuoi
python -m src.compare_models 3     # so sanh 5 kien truc (3 hat giong)
streamlit run app/app.py           # ung dung ky thuat

```

CHƯƠNG B. CODE SNIPPERS QUAN TRỌNG

B.1. Pinball Loss (train.py)

Listing B.1: Hàm mất mát pinball loss

```

1 def pinball_loss_torch(pred, target, quantiles):
2     """pred: (batch, horizon, Q), target: (batch, horizon)"""
3     target = target.unsqueeze(-1)          # (batch, horizon
4         , 1)
5     errors = target - pred
6     q = quantiles.view(1, 1, -1)          # broadcast
7     loss = torch.maximum(q * errors, (q - 1.0) * errors)
8     return loss.mean()

```

B.2. Đầu ra đơn điệu N-BEATS (nbeats.py)

Listing B.2: Tham số hóa đầu ra đa phân vị không crossing

```

1 def forward(self, x):
2     forecast = self.fc_fore(theta)          # (batch, H*Q)
3     forecast = forecast.view(batch, H, Q)
4     base = forecast[..., :1]                # P5
5     increments = F.softplus(forecast[..., 1:])
6     # cumsum đảm bảo tang đơn điệu: P5 <= P10 <= ... <= P95
7     return torch.cat([base, base + torch.cumsum(increments,
8         dim=-1)], dim=-1)

```

B.3. Newsvendor (inventory.py)

Listing B.3: Mô hình Newsvendor từ phân vị rời rạc

```

1 def newsvendor(quantile_levels, quantile_values,
2     underage_cost, overage_cost,
3     current_position=0.0) -> NewsvendorResult:
4     cr = underage_cost / (underage_cost + overage_cost)

```

```

5     target = float(np.interp(cr, quantile_levels,
6                               quantile_values))
7     # Chi phí kỳ vọng qua tích phân xấp xỉ (999 điểm)
8     dense_p = np.linspace(0.001, 0.999, 999)
9     dense_d = np.interp(dense_p, quantile_levels,
10                        quantile_values)
11     short = np.clip(dense_d - target, 0, None).mean() *
12             underage_cost
13     over  = np.clip(target - dense_d, 0, None).mean() *
14             overage_cost
15     return NewsvendorResult(
16         critical_ratio=cr, target_stock=target,
17         order_quantity=max(0.0, target - current_position),
18         expected_understock_cost=float(short),
19         expected_overstock_cost=float(over), ...
20     )

```

B.4. RevIN Denormalize 3D (revin.py)

Listing B.4: Denormalize tương thích tensor 3D

```

1 def denormalize(self, x: torch.Tensor) -> torch.Tensor:
2     if self.affine:
3         x = (x - self.beta) / (self.gamma + self.eps)
4         # Broadcast: (batch, 1, 1) * (batch, horizon, Q)
5         shape = (x.shape[0],) + (1,) * (x.dim() - 1)
6         return x * self._std.view(shape) + self._mean.view(shape)

```

B.5. NHITS block tự cài đặt (models.py)

Listing B.5: Block NHITS - lấy mẫu đa tỷ lệ + noi suy

```

1 class _NHITSBlock(nn.Module):
2     def __init__(self, lookback, horizon, n_quantiles, hidden
3         ,
4         n_layers, dropout, pool_kernel):
5         super().__init__()

```

```
5         self.pool = nn.MaxPool1d(pool_kernel, stride=
           pool_kernel, ceil_mode=True)
6         pooled_len = -(-lookback // pool_kernel)      #
           ceil
7         self.mlp = _mlp(pooled_len, hidden, n_layers, dropout
           )
8         self.backcast_head = nn.Linear(hidden, pooled_len)
9         self.forecast_head = nn.Linear(hidden, horizon *
           n_quantiles)
10
11     def forward(self, x):                                # x:
           (B, L)
12         pooled = self.pool(x.unsqueeze(1)).squeeze(1)  # ha
           mau
13         h = self.mlp(pooled)
14         bc_low = self.backcast_head(h).unsqueeze(1)
15         backcast = F.interpolate(bc_low, size=self.lookback,
16                                 mode="linear").squeeze(1) #
           noi suy
17         return backcast, self.forecast_head(h)
```


CHƯƠNG C. KHAI BÁO SỬ DỤNG AI

Nhóm có sử dụng công cụ AI hỗ trợ trong quá trình thực hiện, tuân thủ quy định của học phần. Chi tiết ở Bảng C.1. Nhóm **chịu trách nhiệm hoàn toàn** về tính chính xác của nội dung, kết quả phân tích và khuyến nghị; mọi kết quả số đều được kiểm chứng bằng cách chạy lại mã nguồn.

Bảng C.1: Khai báo công cụ AI đã sử dụng

Công cụ	Mục đích sử dụng	Cách kiểm chứng
Claude (Anthropic)	Gợi ý khung mã, hỗ trợ cài đặt các mô hình so sánh, sửa lỗi cú pháp, soạn thảo L ^A T _E X	Chạy lại toàn bộ pipeline, đối chiếu số liệu với results/
Claude (Anthropic)	Rà soát chính tả, diễn đạt tiếng Việt trong báo cáo	Nhóm đọc lại và chỉnh sửa thủ công

Phần **không** dùng AI tạo tự động: thiết kế bài toán, lựa chọn phương pháp, diễn giải kết quả và các khuyến nghị quyết định - đây là phần do nhóm tự thực hiện dựa trên dữ liệu và kết quả mô hình.

CHƯƠNG D. BẢNG TỰ CHẤM ĐIỂM

Bảng D.1: Bảng nhóm tự đánh giá theo tiêu chí chấm điểm

STT	Tiêu chí	Tối đa	Tự chấm
1	Hiểu vấn đề và xác định bài toán ra quyết định	15	14
2	Dữ liệu và tiền xử lý dữ liệu	15	14
3	Phân tích dữ liệu và phát hiện insight	20	18
4	Mô hình/hệ thống hỗ trợ ra quyết định	20	19
5	Diễn giải kết quả và khuyến nghị quyết định	15	14
6	Chất lượng báo cáo viết và hồ sơ minh chứng	10	9
7	Tổ chức nhóm và đóng góp cá nhân	5	5
	Tổng cộng	100	93

Ghi chú: điểm tự chấm là đề xuất tham khảo, không thay thế điểm của giảng viên.